

Politécnico
Castelo Branco
Escola Superior
de Tecnologia

Master's Degree in Computer Engineering
Business Intelligence Project

Analyzing Community Dynamics: A StackOverflow Case Study using Neo4j AuraDB

Academic Year 2025/2026

Professor
Luís Barata

Student
Luciano Vitale

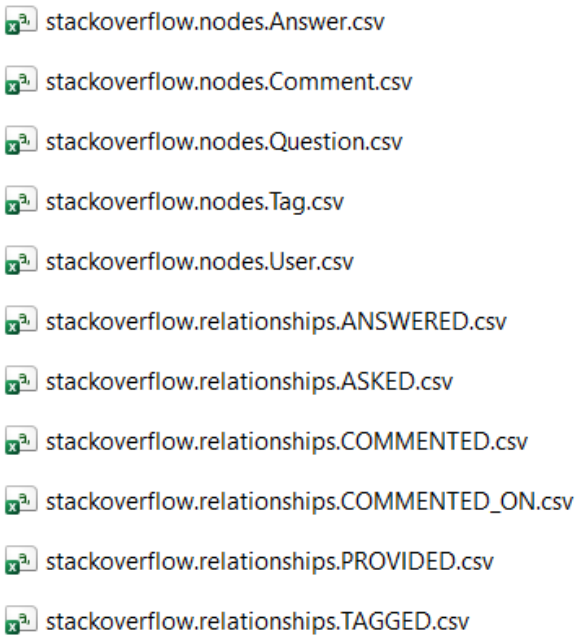
Contents

1. Dataset Origin and Graph Modeling..... 3
2. Data Importation and Processing.....8
3. Database analysis and data enrichment.....16
5. Dashboard Configuration.....23
6. Results and Insight..... 37
7. AI Agent Implementation.....39

1. Dataset Origin and Graph Modeling

The dataset used in this project is the StackOverflow Community Dataset, sourced from the official Neo4j Graph Examples repository on GitHub. This dataset was selected because it represents a complex, highly interconnected ecosystem of users that reflect the real-world interactions within the StackOverflow platform.

The source data is provided in CSV format and is categorized into two primary types:



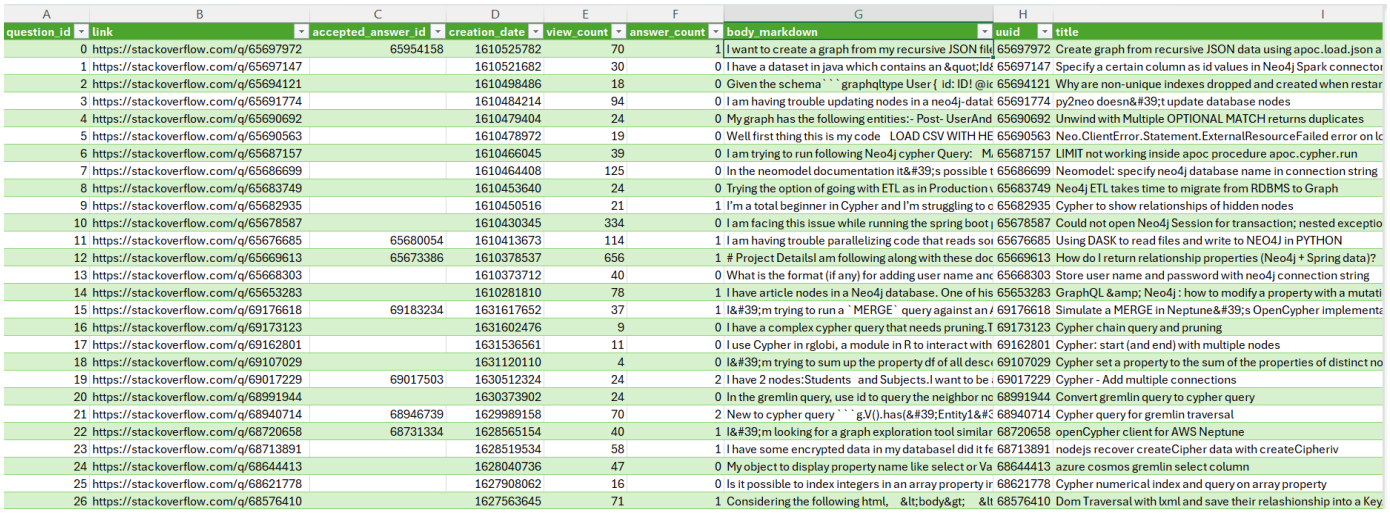
Nodes:
Individual files containing the core entities, such as Users, Questions, Answers, Comments, and Tags.

Relationships:
Specific files that define the logical connections between these entities

The complete dataset and its documentation can be accessed via the following link: https://github.com/neo4j-graph-examples/stackoverflow/tree/main.

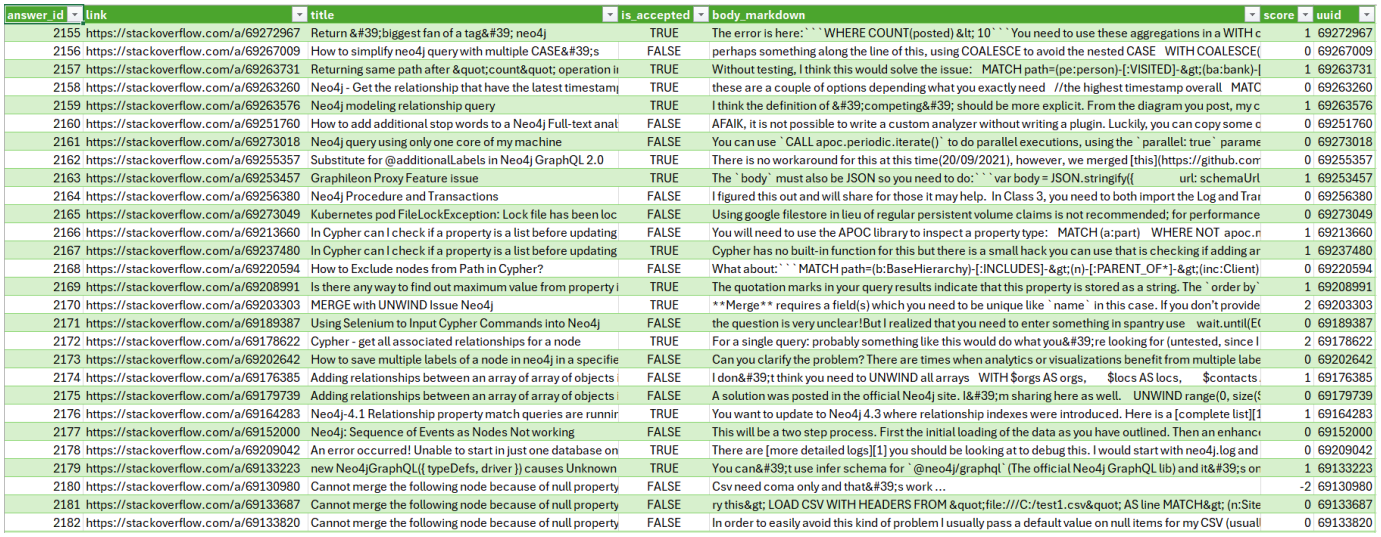
1. Node Data Files

QuestionCSV: Stores the main threads of Stack Overflow. It contains the unique `question_id`, the text body (`body_markdown`), title, creation date, and metrics like view and answer counts.



question_id	link	accepted_answer_id	creation_date	view_count	answer_count	body_markdown	uuid	title
0	https://stackoverflow.com/q/65697972	65954158	1610525782	70	1	I want to create a graph from my recursive JSON file	65697972	Create graph from recursive JSON data using apoc.load.json a
1	https://stackoverflow.com/q/65697147		1610521682	30	0	I have a dataset in java which contains an "Id&	65697147	Specify a certain column as id values in Neo4j Spark connector
2	https://stackoverflow.com/q/65694121		1610498486	18	0	Given the schema```graphqltype User { id: ID! @id	65694121	Why are non-unique indexes dropped and created when restar
3	https://stackoverflow.com/q/65691774		1610484214	94	0	I am having trouble updating nodes in a neo4j-datat	65691774	py2neo doesn't update database nodes
4	https://stackoverflow.com/q/65690692		1610479404	24	0	My graph has the following entities:- Post- UserAnd	65690692	Unwind with Multiple OPTIONAL MATCH returns duplicates
5	https://stackoverflow.com/q/65690563		1610478972	19	0	Well first thing this is my code LOAD CSV WITH HE	65690563	Neo.ClientError.Statement.ExternalResourceFailed error on lo
6	https://stackoverflow.com/q/65687157		1610466045	39	0	I am trying to run following Neo4j cypher Query: M	65687157	LIMIT not working inside apoc procedure apoc.cypher.run
7	https://stackoverflow.com/q/65686699		1610464408	125	0	In the neomodel documentation it's possible t	65686699	Neomodel: specify neo4j database name in connection string
8	https://stackoverflow.com/q/65683749		1610453640	24	0	Trying the option of going with ETL as in Production \	65683749	Neo4j ETL takes time to migrate from RDBMS to Graph
9	https://stackoverflow.com/q/65682935		1610450516	21	1	I'm a total beginner in Cypher and I'm struggling to c	65682935	Cypher to show relationships of hidden nodes
10	https://stackoverflow.com/q/65678587		1610430345	334	0	I am facing this issue while running the spring boot j	65678587	Could not open Neo4j Session for transaction; nested exceptio
11	https://stackoverflow.com/q/65676685	65680054	1610413673	114	1	I am having trouble parallelizing code that reads so	65676685	Using DASK to read files and write to NEO4J in PYTHON
12	https://stackoverflow.com/q/65669613	65673386	1610378537	656	1	# Project DetailsI am following along with these doc	65669613	How do I return relationship properties (Neo4j + Spring data)?
13	https://stackoverflow.com/q/65668303		1610373712	40	0	What is the format (if any) for adding user name anc	65668303	Store user name and password with neo4j connection string
14	https://stackoverflow.com/q/65653283		1610281810	78	1	I have article nodes in a Neo4j database. One of his	65653283	GraphQL & Neo4j : how to modify a property with a mutati
15	https://stackoverflow.com/q/69176618	69183234	1631617652	37	1	I'm trying to run a `MERGE` query against an A	69176618	Simulate a MERGE in Neptune's OpenCypher implementa
16	https://stackoverflow.com/q/69173123		1631602476	9	0	I have a complex cypher query that needs pruning.T	69173123	Cypher chain query and pruning
17	https://stackoverflow.com/q/69162801		1631536561	11	0	I use Cypher in rglobi, a module in R to interact with	69162801	Cypher: start (and end) with multiple nodes
18	https://stackoverflow.com/q/69107029		1631120110	4	0	I'm trying to sum up the property df of all desc	69107029	Cypher set a property to the sum of the properties of distinct no
19	https://stackoverflow.com/q/69017229	69017503	1630512324	24	2	I have 2 nodes:Students and Subjects.I want to be a	69017229	Cypher - Add multiple connections
20	https://stackoverflow.com/q/68991944		1630373902	24	0	In the gremlin query, use id to query the neighbor nc	68991944	Convert gremlin query to cypher query
21	https://stackoverflow.com/q/68940714	68946739	1629989158	70	2	New to cypher query ```g.V().has('Entity1&#3	68940714	Cypher query for gremlin traversal
22	https://stackoverflow.com/q/68720658	68731334	1628565154	40	1	I'm looking for a graph exploration tool similar	68720658	openCypher client for AWS Neptune
23	https://stackoverflow.com/q/68713891		1628519534	58	1	I have some encrypted data in my database I did it fe	68713891	nodejs recover createCipher data with createCipheriv
24	https://stackoverflow.com/q/68644413		1628040736	47	0	My object to display property name like select or Va	68644413	azure cosmos gremlin select column
25	https://stackoverflow.com/q/68621778		1627908062	16	0	Is it possible to index integers in an array property ir	68621778	Cypher numerical index and query on array property
26	https://stackoverflow.com/q/68576410		1627563645	71	1	Considering the following html, <body> &lt	68576410	Dom Traversal with lxml and save their relashionship into a Key

answerCSV: Holds the responses provided by users. It includes the `answer_id`, the markdown text, the user's score, and a crucial boolean flag (`is_accepted`) indicating if it was marked as the correct solution.



answer_id	link	title	is_accepted	body_markdown	score	uuid
2155	https://stackoverflow.com/a/69272967	Return 'biggest fan of a tag' neo4j	TRUE	The error is here: ```WHERE COUNT(posted) < 10``` You need to use these aggregations in a WITH c	1	69272967
2156	https://stackoverflow.com/a/69267009	How to simplify neo4j query with multiple CASE's	FALSE	perhaps something along the line of this, using COALESCE to avoid the nested CASE WITH COALESCE(	0	69267009
2157	https://stackoverflow.com/a/69263731	Returning same path after "count" operation i	TRUE	Without testing, I think this would solve the issue: MATCH path=(pe:person)-[:VISITED]->(ba:bank)-[	1	69263731
2158	https://stackoverflow.com/a/69263260	Neo4j - Get the relationship that have the latest timestamp	TRUE	these are a couple of options depending what you exactly need //the highest timestamp overall MATC	0	69263260
2159	https://stackoverflow.com/a/69263576	Neo4j modeling relationship query	TRUE	I think the definition of 'competing' should be more explicit. From the diagram you post, my c	1	69263576
2160	https://stackoverflow.com/a/69251760	How to add additional stop words to a Neo4j Full-text anal	FALSE	AFAIK, it is not possible to write a custom analyzer without writing a plugin. Luckily, you can copy some c	0	69251760
2161	https://stackoverflow.com/a/69273018	Neo4j query using only one core of my machine	FALSE	You can use `CALL apoc.periodic.iterate()` to do parallel executions, using the `parallel: true` parame	0	69273018
2162	https://stackoverflow.com/a/69255357	Substitute for @additionalLabels in Neo4j GraphQL 2.0	TRUE	There is no workaround for this at this time(20/09/2021), however, we merged [this](https://github.com	0	69255357
2163	https://stackoverflow.com/a/69253457	Graphileon Proxy Feature issue	TRUE	The `body` must also be JSON so you need to do:```var body = JSON.stringify({ url: schemaUrl	1	69253457
2164	https://stackoverflow.com/a/69256380	Neo4j Procedure and Transactions	FALSE	I figured this out and will share for those it may help. In Class 3, you need to both import the Log and Trar	0	69256380
2165	https://stackoverflow.com/a/69273049	Kubernetes pod FileLockException: Lock file has been loc	FALSE	Using google filestore in lieu of regular persistent volume claims is not recommended; for performance	0	69273049
2166	https://stackoverflow.com/a/69213660	In Cypher can I check if a property is a list before updating	FALSE	You will need to use the APOC library to inspect a property type: MATCH (a:part) WHERE NOT apoc.n	1	69213660
2167	https://stackoverflow.com/a/69237480	In Cypher can I check if a property is a list before updating	TRUE	Cypher has no built-in function for this but there is a small hack you can use that is checking if adding ar	1	69237480
2168	https://stackoverflow.com/a/69220594	How to Exclude nodes from Path in Cypher?	FALSE	What about:```MATCH path=(b:BaseHierarchy)-[:INCLUDES]->(n)-[:PARENT_OF*]->(inc:Client)	0	69220594
2169	https://stackoverflow.com/a/69208991	Is there any way to find out maximum value from property i	TRUE	The quotation marks in your query results indicate that this property is stored as a string. The `order by`	1	69208991
2170	https://stackoverflow.com/a/69203303	MERGE with UNWIND Issue Neo4j	TRUE	**Merge** requires a field(s) which you need to be unique like `name` in this case. If you don't provide	2	69203303
2171	https://stackoverflow.com/a/69189387	Using Selenium to Input Cypher Commands into Neo4j	FALSE	the question is very unclear!But I realized that you need to enter something in spantry use wait.until(EC	0	69189387
2172	https://stackoverflow.com/a/69178622	Cypher - get all associated relationships for a node	TRUE	For a single query: probably something like this would do what you're looking for (untested, since I	2	69178622
2173	https://stackoverflow.com/a/69202642	How to save multiple labels of a node in neo4j in a specifie	FALSE	Can you clarify the problem? There are times when analytics or visualizations benefit from multiple labe	0	69202642
2174	https://stackoverflow.com/a/69176385	Adding relationships between an array of array of objects i	FALSE	I don't think you need to UNWIND all arrays WITH $orgs AS orgs, $locs AS locs, $contacts	1	69176385
2175	https://stackoverflow.com/a/69179739	Adding relationships between an array of array of objects i	FALSE	A solution was posted in the official Neo4j site. I'm sharing here as well. UNWIND range(0, size(s	0	69179739
2176	https://stackoverflow.com/a/69164283	Neo4j-4.1 Relationship property match queries are runnir	TRUE	You want to update to Neo4j 4.3 where relationship indexes were introduced. Here is a [complete list][1	1	69164283
2177	https://stackoverflow.com/a/69152000	Neo4j: Sequence of Events as Nodes Not working	FALSE	This will be a two step process. First the initial loading of the data as you have outlined. Then an enhanc	0	69152000
2178	https://stackoverflow.com/a/69209042	An error occurred! Unable to start in just one database on	TRUE	There are [more detailed logs][1] you should be looking at to debug this. I would start with neo4j.log and	0	69209042
2179	https://stackoverflow.com/a/69133223	new Neo4jGraphQL({ typeDefs, driver }) causes Unknown	TRUE	You can't use infer schema for `@neo4j/graphql` (The official Neo4j GraphQL lib) and it's or	1	69133223
2180	https://stackoverflow.com/a/69130980	Cannot merge the following node because of null property	FALSE	Csv need coma only and that's work ...	-2	69130980
2181	https://stackoverflow.com/a/69133687	Cannot merge the following node because of null property	FALSE	ry this> LOAD CSV WITH HEADERS FROM "file:///C:/test1.csv" AS line MATCH> (n:Site	0	69133687
2182	https://stackoverflow.com/a/69133820	Cannot merge the following node because of null property	FALSE	In order to easily avoid this kind of problem I usually pass a default value on null items for my CSV (usuall	0	69133820

commentCSV: Contains the brief remarks left on posts. It stores the `comment_id`, the comment's score, and a direct URL link to the comment.

comment_id	score	uuid	link
5943	0	122336972	https://stackoverflow.com/questions/69208813/configuring-and-connect-to-neo4j#comment122336972_69208813
5944	0	122344431	https://stackoverflow.com/questions/69208813/configuring-and-connect-to-neo4j#comment122344431_69208813
5945	0	122318244	https://stackoverflow.com/questions/69199782/neo4j-return-n-to-columns-with-distinct-keysn-as-headers#comment122318244_69199782
5946	0	122324738	https://stackoverflow.com/questions/69199782/neo4j-return-n-to-columns-with-distinct-keysn-as-headers#comment122324738_69199782
5947	1	122267204	https://stackoverflow.com/questions/69177409/how-to-store-knowledge-graph#comment122267204_69177409
5948	0	122269010	https://stackoverflow.com/questions/69177409/how-to-store-knowledge-graph#comment122269010_69177409
5949	0	122369050	https://stackoverflow.com/questions/69177409/how-to-store-knowledge-graph#comment122369050_69177409
5950	1	122315777	https://stackoverflow.com/questions/69163777/connection-closed-accessing-neo4j-in-docker-from-node-js-in-docker-on-mac#comment122315777_69163777
5951	0	122321125	https://stackoverflow.com/questions/69163777/connection-closed-accessing-neo4j-in-docker-from-node-js-in-docker-on-mac#comment122321125_69163777
5952	0	122234450	https://stackoverflow.com/questions/69158730/neo4j-apoc-procedure-do-not-import-all-of-my-data#comment122234450_69158730
5953	0	122239749	https://stackoverflow.com/questions/69158730/neo4j-apoc-procedure-do-not-import-all-of-my-data#comment122239749_69158730
5954	0	122215334	https://stackoverflow.com/questions/69147992/not-able-to-create-graph-with-native-projection#comment122215334_69147992
5955	0	122234327	https://stackoverflow.com/questions/69147992/not-able-to-create-graph-with-native-projection#comment122234327_69147992
5956	0	122234368	https://stackoverflow.com/questions/69147992/not-able-to-create-graph-with-native-projection#comment122234368_69147992
5957	0	122234668	https://stackoverflow.com/questions/69147992/not-able-to-create-graph-with-native-projection#comment122234668_69147992
5958	1	122235683	https://stackoverflow.com/questions/69147992/not-able-to-create-graph-with-native-projection#comment122235683_69147992
5959	0	122187069	https://stackoverflow.com/questions/69118543/spring-data-neo4j-field-annotation-validation#comment122187069_69118543
5960	0	122187325	https://stackoverflow.com/questions/69118543/spring-data-neo4j-field-annotation-validation#comment122187325_69118543
5961	0	122177158	https://stackoverflow.com/questions/69107479/neo4j-bloom-slows-down-with-increased-number-of-labels#comment122177158_69107479
5962	1	122138207	https://stackoverflow.com/questions/69104919/py2neo-indexerror-index-out-of-range-on-graph-run-with-gds-alpha-allshortestpat#comment122138207_69104919
5963	0	122127469	https://stackoverflow.com/questions/69098868/shortest-path-between-node-a-and-first-occurance-of-node-b#comment122127469_69098868
5964	0	122113415	https://stackoverflow.com/questions/69084096/inheritance-of-parent-ontology-attributes-in-neo4j#comment122113415_69084096
5965	0	122264119	https://stackoverflow.com/questions/69084096/inheritance-of-parent-ontology-attributes-in-neo4j#comment122264119_69084096
5966	0	121871587	https://stackoverflow.com/questions/68956342/neo4j-load-csv-a-gzip-file-from-ftp#comment121871587_68956342
5967	0	121892273	https://stackoverflow.com/questions/68956342/neo4j-load-csv-a-gzip-file-from-ftp#comment121892273_68956342
5968	0	121955804	https://stackoverflow.com/questions/68956342/neo4j-load-csv-a-gzip-file-from-ftp#comment121955804_68956342
5969	0	121960662	https://stackoverflow.com/questions/68951239/is-there-an-option-or-a-workaround-to-collapse-a-node-in-neo4j-graphxr-view#comment121960662_68951239
5970	0	121824589	https://stackoverflow.com/questions/68930781/neo4j-install-on-ubuntu-password-not-working#comment121824589_68930781
5971	0	121825152	https://stackoverflow.com/questions/68930781/neo4j-install-on-ubuntu-password-not-working#comment121825152_68930781

user_id	uuid	display_name
604	2236092	schernichkin
605	13993723	Damian Grzanka
606	5243272	sandrooco
607	12152568	Bobis
608	16377721	wanderingcatto
609	13997373	bguegan
610	1839104	Swordfish
611	5773084	Chris Wohlert
612	3507615	zulfi123786
613	15205110	Florian Bauer
614	8598739	Paras Bansal
615	9807545	Rasputin
616	4780428	Yoann Maingon
617	4052014	Marcos
618	5868325	Ahlem rhayem
619	12877858	MASTANELAL ABDULGAFAR KURESHI
620	16922964	sprint_6
621	10768865	Stephanie

userCSV: Represents the profiles of the Stack Overflow community members. It contains their `user_id`, a unique identifier (`uuid`), and their `display_name`.

tag_id	name	link
1679	neo4j	https://stackoverflow.com/questions/tagged/neo4j
1680	subgraph	https://stackoverflow.com/questions/tagged/subgraph
1681	graph	https://stackoverflow.com/questions/tagged/graph
1682	cypher	https://stackoverflow.com/questions/tagged/cypher
1683	graph-databases	https://stackoverflow.com/questions/tagged/graph-databases
1684	neo4j-apoc	https://stackoverflow.com/questions/tagged/neo4j-apoc
1685	database	https://stackoverflow.com/questions/tagged/database
1686	spring-boot	https://stackoverflow.com/questions/tagged/spring-boot
1687	grandstack	https://stackoverflow.com/questions/tagged/grandstack
1688	search	https://stackoverflow.com/questions/tagged/search
1689	full-text-search	https://stackoverflow.com/questions/tagged/full-text-search
1690	parallel-processing	https://stackoverflow.com/questions/tagged/parallel-processing
1691	graphql	https://stackoverflow.com/questions/tagged/graphql
1692	jquery	https://stackoverflow.com/questions/tagged/jquery
1693	ajax	https://stackoverflow.com/questions/tagged/ajax
1694	graphileon	https://stackoverflow.com/questions/tagged/graphileon
1695	java	https://stackoverflow.com/questions/tagged/java
1696	plugins	https://stackoverflow.com/questions/tagged/plugins
1697	docker	https://stackoverflow.com/questions/tagged/docker
1698	kubernetes	https://stackoverflow.com/questions/tagged/kubernetes

tagCSV: Stores the topics used to categorize questions (such as "neo4j", "python", or "cypher"). It includes the `tag_id`, the tag's name, and a web link to that specific tag's page.

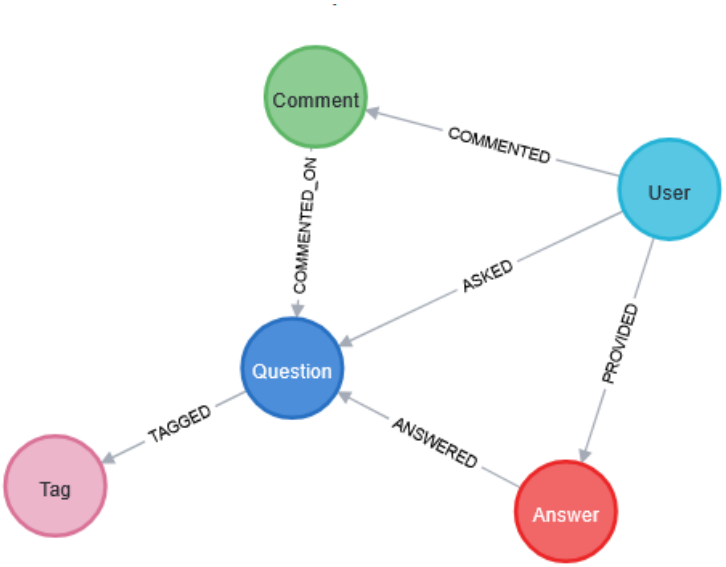
The modeling rationale focuses on transforming tabular CSV data into a connected graph consisting of 5 node types and 6 relationship types:

Nodes (Entities):

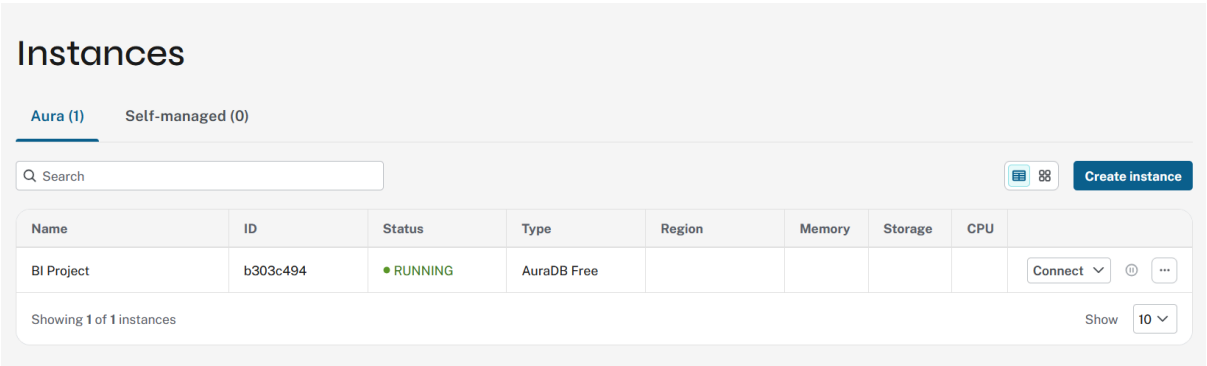
- User: Registered members of the platform.
- Question: Queries posted by users.
- Answer: Solutions or responses provided for specific questions.
- Comment: Additional remarks left on questions or answers.
- Tag: Categorical labels identifying the technologies involved.

Relationships (Connections):

- Asked: Connects a User to a created Question.
- Provided: Connects a User to a specific Answer.
- Answered: Links an Answer to its corresponding Question.
- Commented: Connects a User to a Comment.
- Commented_on: Links a Comment to the Question it addresses.
- Tagged: Associates a Question with its descriptive Tags.

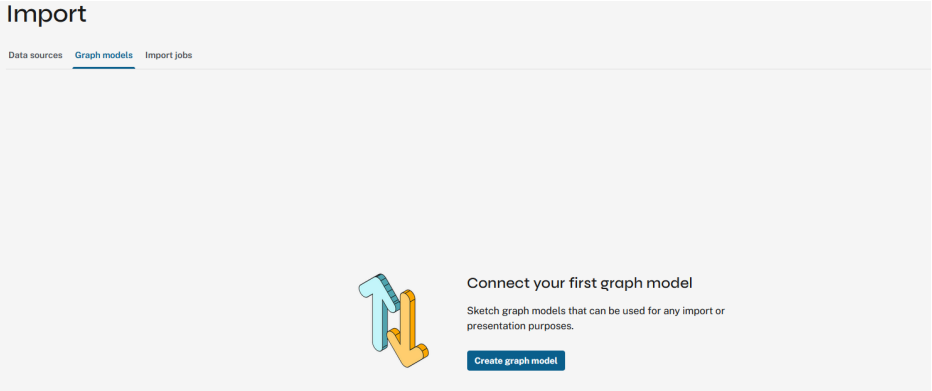


For data processing and analysis, a dedicated cloud instance was configured using Neo4j AuraDB Free. The instance provides the necessary infrastructure to host the graph database, execute Cypher queries.

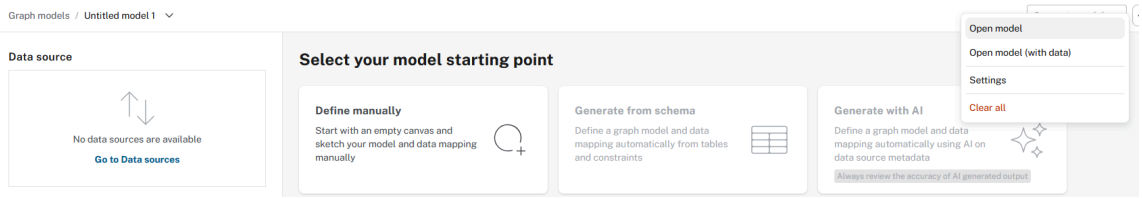

Instances
Aura (1)
Self-managed (0)
Search
Create instance
Name
ID
Status
Type
Region
Memory
Storage
CPU
BI Project
b303c494
RUNNING
AuraDB Free
Connect
Showing 1 of 1 instances
Show
10

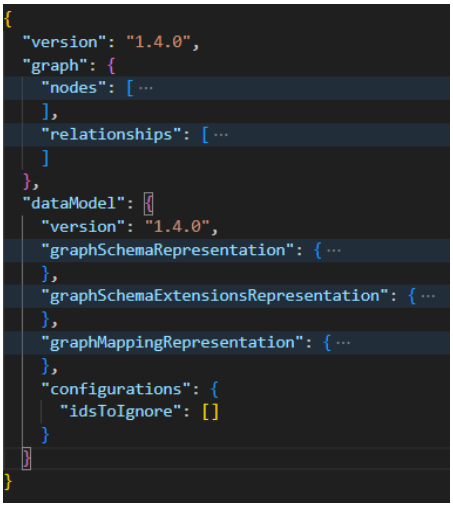
2. Data Importation and Processing

The process began by accessing the Neo4j Importer interface. A new, empty graph project was created to serve as the staging area for defining the model and uploading the data files.



Instead of manually defining nodes and relationships, we leveraged the architectural assets provided in the Neo4j GitHub repository. The `neo4j_importer_model.json` file was opened into the tool. This automatically populated the importer canvas with the theoretical graph model.





The JSON file is organized into two primary sections: graph and dataModel.





The graph Section outlines the logical entities and their spatial layout:

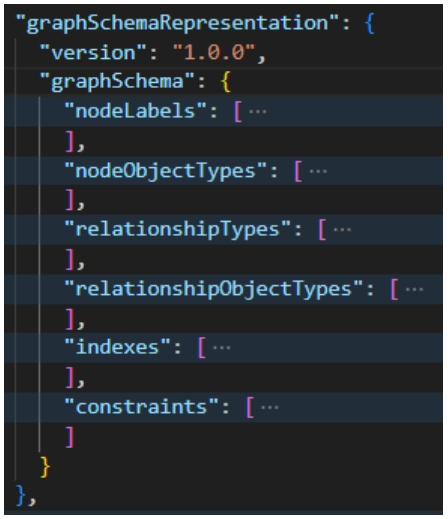
Nodes:

It lists the core entities of the dataset: Question, Answer, Comment, User, and Tag, assigning each an internal ID and `x`, `y` coordinates for spatial visualization.

Relationships:

It defines how these nodes interact with one another. For example, it dictates that an "Answer" node connects to a "Question" node via the `ANSWERED` relationship.

The dataModel Section defines the strict rules for the database and explains how the raw CSV data will be transformed into the conceptual graph. It is broken down into three main components:



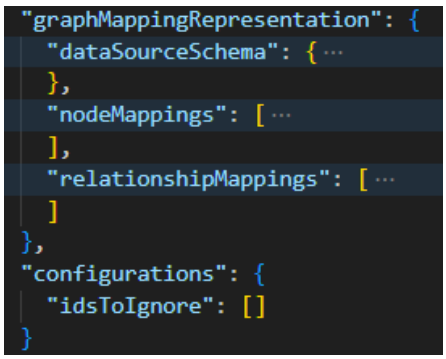
graphSchemaRepresentation:

Defines the database architecture by establishing `nodeLabels` and enforcing strict data types for their `properties` (such as defining `view_count` as an integer and `is_accepted` as a boolean). Crucially, it establishes `indexes` to optimize query performance and applies uniqueness `constraints` to prevent duplicate nodes.



graphSchemaExtensionsRepresentation:

Extends the schema to explicitly declare primary keys, known here as `nodeKeyProperties`, for each specific node type.

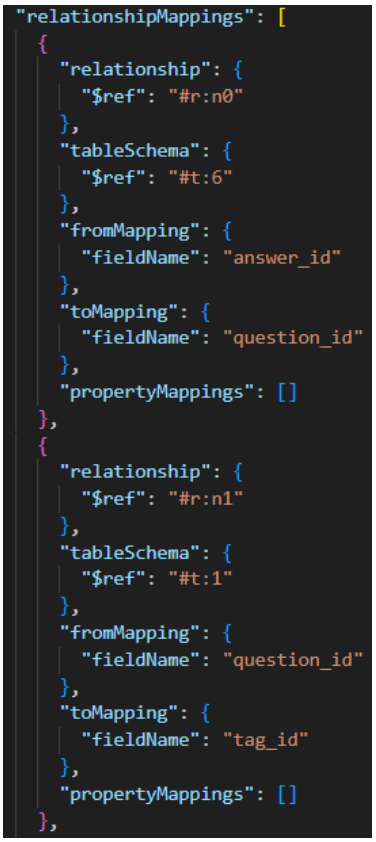


graphMappingRepresentation:

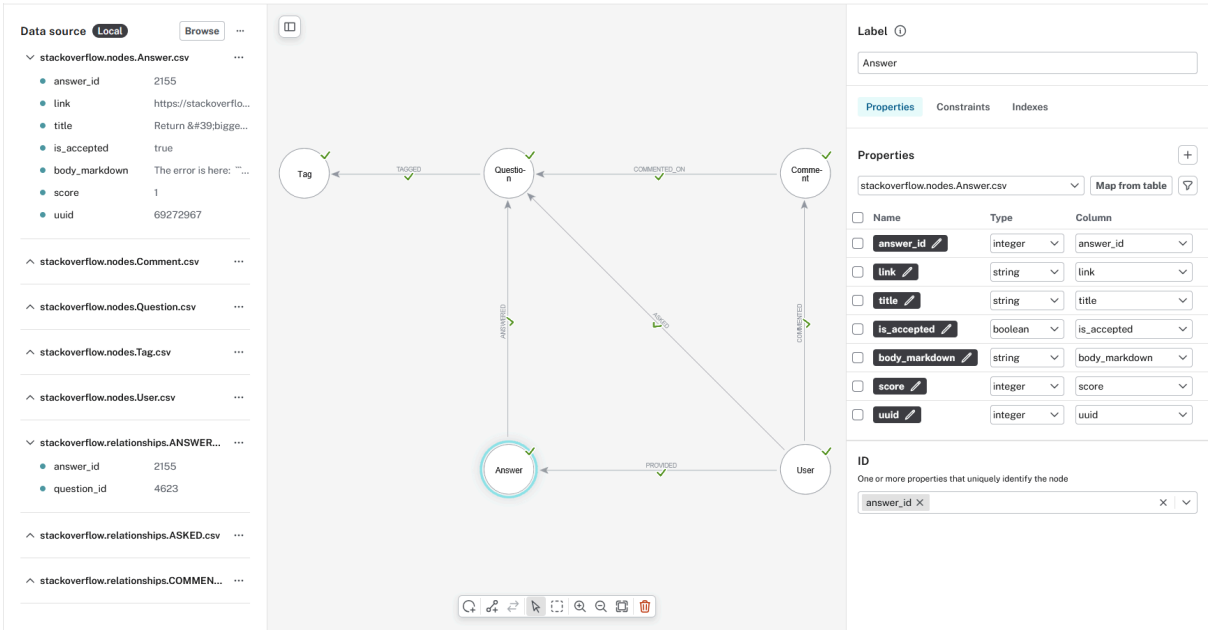
This acts as the bridge linking the raw data to the graph schema. It defines the `dataSourceSchema`, identifying the local CSV files and detailing their exact column fields. It then utilizes `nodeMappings` and `relationshipMappings` to provide precise instructions to the database on how to extract columns from those CSV files and map them to the corresponding predefined node properties and graph relationships.



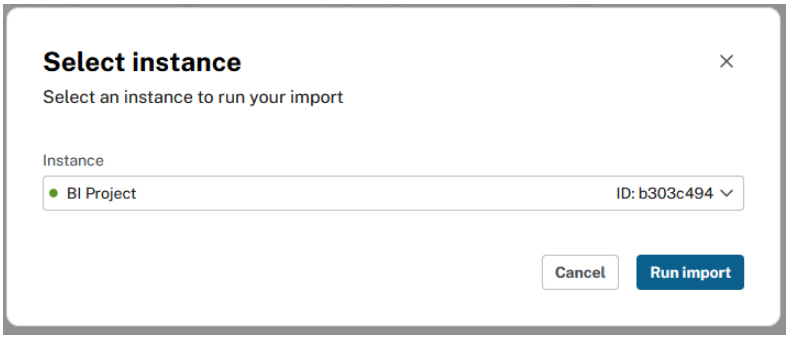




Next, the corresponding CSV files for both nodes and relationships were uploaded into the importer. The tool utilizes the configuration within the JSON model to automatically map each CSV file to its respective node label or relationship type. This step linked the raw tabular data with the defined graph schema.



The final step involved connecting the Neo4j Importer to the running "BI Project" AuraDB instance.



After selecting the instance, the "Run Import" command was executed.

The Importer dynamically generated optimized Cypher commands to populate the database. Before importing any actual data, Neo4j needs to set up the structural rules of the database to guarantee data integrity and query performance. (Applies to the queries for: Question, Answer, Comment, Tag, and User)

- CREATE CONSTRAINT (Key statements): These queries enforce uniqueness on your primary keys. By stating `REQUIRE (n.question_id) IS UNIQUE`, the database guarantees that no two Question nodes can share the same ID. This same safety mechanism is automatically generated for `answer_id`, `comment_id`, `user_id`, and `tag_id`.
- CREATE INDEX: These statements build search indexes on secondary properties that might be frequently searched. For example, it creates indexes for the `uuid` of Answers, Comments, Questions, and Users, as well as for the `name` property of Tags. Indexes act like a book's table of contents, drastically speeding up future data retrieval.

Query examples for Answer and Comment nodes:

Key statement

```
CREATE CONSTRAINT `answer_id_Answer_uniq` IF NOT EXISTS
FOR (n: `Answer`)
REQUIRE (n.`answer_id`) IS UNIQUE;
```

Index statement

```
CREATE INDEX `uuid_Answer` IF NOT EXISTS
FOR (n: `Answer`)
ON (n.`uuid`);
```

Key statement

```
CREATE CONSTRAINT `comment_id_Comment_uniq` IF NOT EXISTS
FOR (n: `Comment`)
REQUIRE (n.`comment_id`) IS UNIQUE;
```

Index statement

```
CREATE INDEX `uuid_Comment` IF NOT EXISTS
FOR (n: `Comment`)
ON (n.`uuid`);
```

Once the schema is secured, the system runs queries to extract data from the CSV files and create the entities.

N.B. The variables starting with a dollar sign ($) in the generated Cypher queries are the query parameters. Because we used an automated JSON, the import tool doesn't hardcode the data directly into the query text. Instead, it creates a highly efficient template and injects the CSV data using these variables.

- $nodeRecords and $relRecords: These variables hold a batch of raw data read directly from our node and relationships CSV files
- $idsToSkip: This parameter acts as a safety filter and is directly linked to the configuration in the JSON file. It instructs the database to ignore specific problematic or unwanted IDs during the import.

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`question_id` IN $idsToSkip AND NOT toInteger(trim(nodeRecord.`question_id`)) IS NULL
MERGE (n: `Question` { `question_id`: toInteger(trim(nodeRecord.`question_id`)) })
SET n.`link` = nodeRecord.`link`
SET n.`accepted_answer_id` = toInteger(trim(nodeRecord.`accepted_answer_id`))
SET n.`creation_date` = toInteger(trim(nodeRecord.`creation_date`))
SET n.`view_count` = toInteger(trim(nodeRecord.`view_count`))
SET n.`answer_count` = toInteger(trim(nodeRecord.`answer_count`))
SET n.`body_markdown` = nodeRecord.`body_markdown`
SET n.`uuid` = toInteger(trim(nodeRecord.`uuid`))
SET n.`title` = nodeRecord.`title`;
```

All the node-loading queries share the exact same structural pattern.

- UNWIND $nodeRecords AS nodeRecord: This command takes a batch of rows from the CSV file and iterates through them one by one.
- WITH * tells the database to pass all currently active variables
- WHERE NOT acts as a strict data filter that prevents bad data from entering the graph by performing two quick safety checks: It drops any row whose ID is found in your $idsToSkip blacklist and blocks rows with missing or NULL primary keys.
- MERGE: Acts as a smart "find or create" function. It looks for an existing node with that specific ID; if it doesn't exist, it creates it. This completely prevents the creation of duplicate nodes.

- SET: After the node is identified or created, the `SET` commands attach the properties to it. During this step, the query cleans and casts the data types on the fly.

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`tag_id` IN $idsToSkip AND NOT toInteger(trim(nodeRecord.`tag_id`)) IS NULL
MERGE (n: `Tag` { `tag_id`: toInteger(trim(nodeRecord.`tag_id`)) })
SET n.`name` = nodeRecord.`name`
SET n.`link` = nodeRecord.`link`;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`answer_id` IN $idsToSkip AND NOT toInteger(trim(nodeRecord.`answer_id`)) IS NULL
MERGE (n: `Answer` { `answer_id`: toInteger(trim(nodeRecord.`answer_id`)) })
SET n.`link` = nodeRecord.`link`
SET n.`title` = nodeRecord.`title`
SET n.`is_accepted` = toLower(trim(nodeRecord.`is_accepted`)) IN ['1','true','yes']
SET n.`body_markdown` = nodeRecord.`body_markdown`
SET n.`score` = toInteger(trim(nodeRecord.`score`))
SET n.`uuid` = toInteger(trim(nodeRecord.`uuid`));
```

After all nodes populate the graph, the final step is to connect them. All relationship queries use a simple but highly effective two-step logic (Applies to the load statements for: Answered, Asked, Commented, Commented_on, Provided and Tagged):

- MATCH (source...) and MATCH (target...): First, the query finds the two specific nodes that need to be connected by looking up their primary keys. For example, to create the `ASKED` relationship, it first locates the specific `User` and the specific `Question`.
- MERGE (source)-[r:RELATIONSHIP_NAME]->(target): Once the endpoints are pinned down, the system draws the directional connection between them

Query examples for Answered and Tagged relationships:

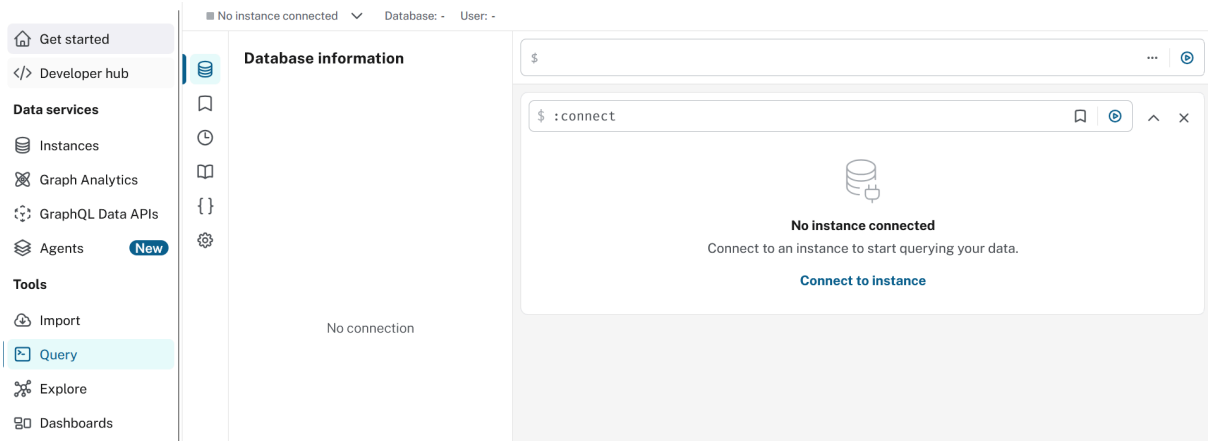
Load statement

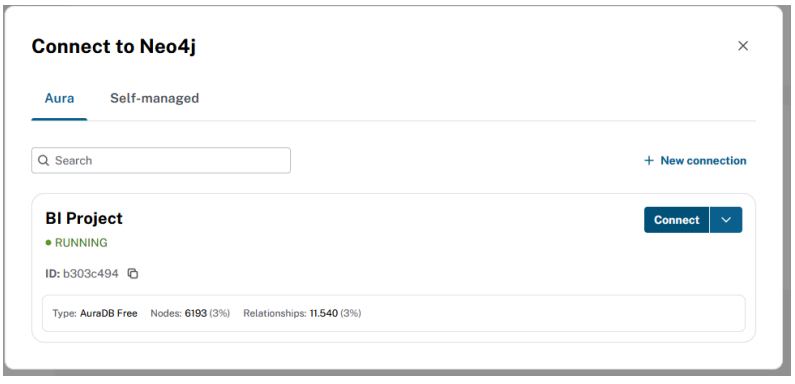
```
UNWIND $relRecords AS relRecord
MATCH (source: `Answer` { `answer_id`: toInteger(trim(relRecord.`answer_id`)) })
MATCH (target: `Question` { `question_id`: toInteger(trim(relRecord.`question_id`)) })
MERGE (source)-[r: `ANSWERED`]→(target);
```

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `User` { `user_id`: toInteger(trim(relRecord.`user_id`)) })
MATCH (target: `Question` { `question_id`: toInteger(trim(relRecord.`question_id`)) })
MERGE (source)-[r: `ASKED`]→(target);
```

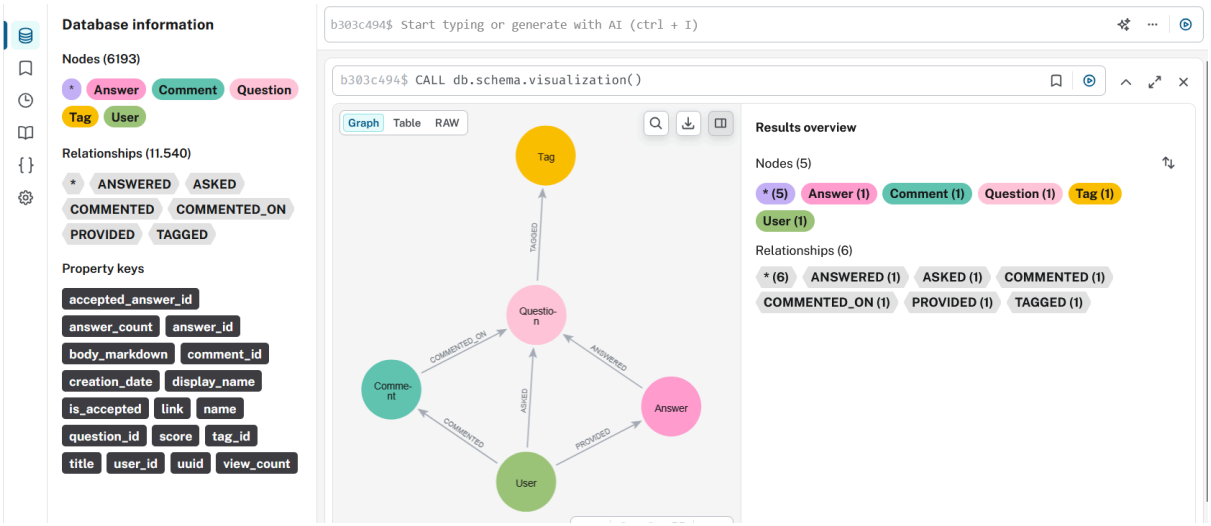
The final step was the transition to the Neo4j database interface to establish a direct connection to the active AuraDB instance.





3. Database analysis and data enrichment

This section focuses on the comprehensive analysis of the database. As an initial step, the query `CALL db.schema.visualization()` was executed to obtain a general overview of the database. This provided key insights into the graph structure, including the number and types of nodes, their properties, and the relationships connecting them. Subsequently, specific Cypher queries were developed to extract actionable data, such as counts, rankings, and aggregated metrics, designed specifically to populate interactive dashboards.



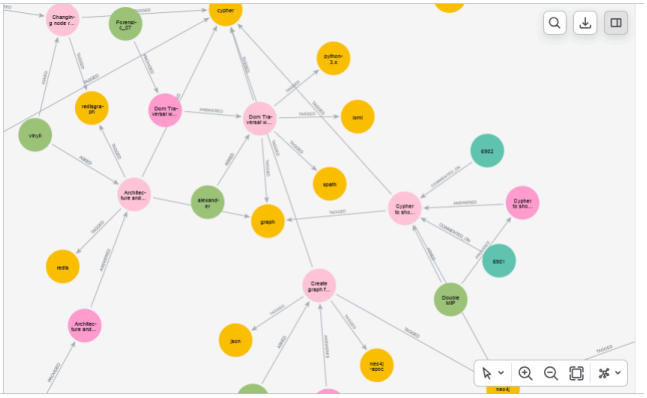
To examine the dataset in greater depth, a second query was executed. This operation retrieved a random sample of nodes along with their corresponding relationships, facilitating a thorough verification of both the relationship validity and the specific fields within each node. The core construct utilized in this query is the `OPTIONAL MATCH` clause, which functions as a non-mandatory pattern match. Neo4j attempts to locate the specified relationship; however, if it is not found, the query continues its execution rather than excluding the main result. The accompanying figure illustrates this process, highlighting the `Question` and `Tag` nodes as examples.

```cypher
// 1. Take 15 questions that have at least one answer to ensure a rich graph
MATCH (q:Question)
WHERE q.answer_count > 0
WITH q LIMIT 15

// 2. Extract all paths (relationships and nodes) directly connected to these
15 questions
OPTIONAL MATCH p_ask = (u:User)-[:ASKED]->(q)
OPTIONAL MATCH p_tag = (q)-[:TAGGED]->(t:Tag)
OPTIONAL MATCH p_ans = (a:Answer)-[:ANSWERED]->(q)
OPTIONAL MATCH p_com = (c:Comment)-[:COMMENTED_ON]->(q)

// 3. Extract the users who provided answers to those questions
OPTIONAL MATCH p_prov = (u2:User)-[:PROVIDED]->(a)

// 4. Return the complete paths
RETURN p_ask, p_tag, p_ans, p_com, p_prov
```



	p_ask	p_tag	p_ans	p_com	p_prov
1	Anu - ASKED → Create graph from recurs...	Create graph from recurs... - TAGGED → neo4j	Create graph from recurs... - ANSWERED → Create graph from recurs...	null	Anu - PROVIDED → Create graph from recurs...
2	Anu - ASKED → Create graph from recurs...	Create graph from recurs... - TAGGED → cypher	Create graph from recurs... - ANSWERED → Create graph from recurs...	null	Anu - PROVIDED → Create graph from recurs...
3	Anu - ASKED →	Create graph from recurs...	Create graph from recurs... -	null	Anu - PROVIDED →

Started streaming 58 r

Node details

Question

Key	Value
<id>	4:156d2408-2966-496d-bdd7-2f5a7be4048f:0
accepted_answer_id	65954158
answer_count	1

body_markdown	"I want to create a graph from my recursive JSON file. My JSON data is given below.
	
	``{
	"ChildNodes":[
	{
	"ChildNodes... Show all

creation_date	1610525782

Node details

Tag

Key	Value
<id>	4:156d2408-2966-496d-bdd7-2f5a7be4048f:5717
link	"https://stackoverflow.com/questions/tagged/neo4j"
name	"neo4j"
tag_id	1679

Following the initial examination, a Data Enrichment phase was initiated. During this stage, queries were executed to prepare the database by appending essential labels and properties required for subsequent analyses. The primary objective was to understand the activity levels of the platform's users. To achieve this, a specific query was performed to evaluate the percentiles of the activity level distribution. This operation utilized APOC aggregation functions—advanced aggregation utilities provided by the APOC library for Neo4j. These functions significantly extend Cypher's native aggregation capabilities (such as `count`, `avg`,

and `sum`) by introducing more powerful statistical, collection, and graph-processing operations.

```
MATCH (u:User)
OPTIONAL MATCH (u)-[:ASKED]->(q:Question)
OPTIONAL MATCH (u)-[:PROVIDED]->(a:Answer)
OPTIONAL MATCH (u)-[:COMMENTED]->(c:Comment)

WITH u, count(DISTINCT q) + count(DISTINCT a) + count(DISTINCT c) AS total_activity

WHERE total_activity > 0

RETURN
  count(u) AS TotalActiveUsers,
  min(total_activity) AS Minimum,
  max(total_activity) AS Maximum,
  round(avg(total_activity), 2) AS Average,
  apoc.agg.percentiles(total_activity, [0.50, 0.75, 0.90, 0.95, 0.99]) AS Percentiles
```

Table RAW

TotalActiveUsers	Minimum	Maximum	Average	Percentiles
1365	1	318	3.19	[1, 2, 5, 7, 20]

Having established these thresholds, we then classified the users based on their activity levels:

```
MATCH (u:User)
OPTIONAL MATCH (u)-[:ASKED]->(q:Question)
OPTIONAL MATCH (u)-[:PROVIDED]->(a:Answer)
OPTIONAL MATCH (u)-[:COMMENTED]->(c:Comment)

WITH u,
     count(DISTINCT q) + count(DISTINCT a) + count(DISTINCT c) AS
total_activity

// Remove old labels
REMOVE u:PowerUser, u:RegularUser, u:CasualUser

// Assign new labels
FOREACH (_ IN CASE WHEN total_activity > 7 THEN [1] ELSE [] END | SET
u:PowerUser)
FOREACH (_ IN CASE WHEN total_activity > 2 AND total_activity <= 7 THEN [1]
ELSE [] END | SET u:RegularUser)
FOREACH (_ IN CASE WHEN total_activity <= 2 THEN [1] ELSE [] END | SET
u:CasualUser)

RETURN
  u.display_name AS user,
  total_activity,
  CASE
    WHEN u:PowerUser THEN "PowerUser"
    WHEN u:RegularUser THEN "RegularUser"
    WHEN u:CasualUser THEN "CasualUser"
  END AS engagement_class

ORDER BY total_activity DESC
```

user	total_activity	engagement_class
"cybersam"	318	"PowerUser"
"Tomaž Bratanič"	193	"PowerUser"
"jose_bacoy"	178	"PowerUser"
"InverseFalcon"	144	"PowerUser"
"fbiville"	138	"PowerUser"
"Graphileon"	135	"PowerUser"

To implement this classification, a specific workaround was required, as Cypher lacks a native `IF` statement for write operations such as `SET`. The solution involves using a `CASE` statement to evaluate the condition and generate a list: `[1]` if the condition is true, or an empty list `[]` if it is false. A `FOREACH` clause then iterates over this list, executing the `SET` operation exactly once if it encounters `[1]`, or skipping the operation entirely if the list is empty.

After determining the user activity levels, the focus shifted to analyzing the actual quality of that activity. For the `ProblemSolver` category, consistent users—defined as those with at least three provided answers—were isolated. A performance score was calculated by multiplying their average vote by a bonus based on their accepted answers. Ultimately, the `ProblemSolver` label was assigned exclusively to the top 10% elite of this group (specifically, those with a score >= 1.875).

```
MATCH (u:User)-[:PROVIDED]->(a:Answer)
WITH u,
     count(a) AS total_answers,
     avg(a.score) AS avg_score,
     sum(CASE WHEN a.is_accepted = true THEN 1 ELSE 0 END) AS accepted_answers

WHERE total_answers >= 3

WITH u,
     (avg_score * (1 + (toFloat(accepted_answers) / total_answers))) AS answer_quality_score

RETURN
     percentileCont(answer_quality_score, 0.5) AS Mediana,
     percentileCont(answer_quality_score, 0.75) AS Top_25_Percent,
     percentileCont(answer_quality_score, 0.90) AS Top_10_Percent,
     percentileCont(answer_quality_score, 0.99) AS Top_1_Percent
```

Table RAW

Mediana	Top_25_Percent	Top_10_Percent	Top_1_Percent
0.8888888888888888	1.1679579889807163	1.875	2.3011937557392104

```
MATCH (u:User)-[:PROVIDED]->(a:Answer)
WITH u,
     count(a) AS total_answers,
     avg(a.score) AS avg_score,
     sum(CASE WHEN a.is_accepted = true THEN 1 ELSE 0 END) AS accepted_answers
WHERE total_answers >= 3
WITH u,
     (avg_score * (1 + (toFloat(accepted_answers) / total_answers))) AS answer_quality_score

SET u.answer_quality_score = round(answer_quality_score, 2)
REMOVE u:ProblemSolver
FOREACH (_ IN CASE WHEN answer_quality_score >= 1.875 THEN [1] ELSE [] END |
    SET u:ProblemSolver
)

RETURN
    count(u) AS UserAnalyzed,
    sum(CASE WHEN answer_quality_score >= 1.875 THEN 1 ELSE 0 END) AS NewProblemSolver
```

Table RAW

UserAnalyzed	NewProblemSolver
51	6

For the `TrendSetter` category, the analysis measured the ability of users to anticipate community needs and generate valuable traffic through their questions. First, consistent users—defined as those who had posted at least two questions—were isolated. A score was then calculated balancing the total number of views (applying logarithmic smoothing to avoid disproportionately rewarding random "viral" spikes) against the average views per question. Finally, the `TrendSetter` label was assigned exclusively to the top 10% elite, corresponding to users who surpassed the mathematical threshold of 254.86.

```
MATCH (u:User)-[:ASKED]->(q:Question)
WITH u,
     count(q) AS total_questions,
     sum(q.view_count) AS total_views,
     avg(q.view_count) AS avg_views

// 25% of total
WHERE total_questions >= 2

WITH u,
     (log10(total_views + 1) * avg_views) AS question_quality_score

RETURN
     percentileCont(question_quality_score, 0.5) AS Mediana,
     percentileCont(question_quality_score, 0.75) AS Top_25_Percent,
     percentileCont(question_quality_score, 0.90) AS Top_10_Percent,
     percentileCont(question_quality_score, 0.99) AS Top_1_Percent
```

Table RAW

Mediana	Top_25_Percent	Top_10_Percent	Top_1_Percent
86.06499992971756	147.8627062542757	254.86089775952985	1138.0220647733481

```
MATCH (u:User)-[:ASKED]->(q:Question)
WITH u,
     count(q) AS total_questions,
     sum(q.view_count) AS total_views,
     avg(q.view_count) AS avg_views
WHERE total_questions >= 2
WITH u,
     (log10(total_views + 1) * avg_views) AS question_quality_score

SET u.question_quality_score = round(question_quality_score, 2)
REMOVE u:TrendSetter
FOREACH (_ IN CASE WHEN question_quality_score >= 254.86 THEN [1] ELSE [] END |
    SET u:TrendSetter
)

RETURN
    count(u) AS AnalyzedUser,
    sum(CASE WHEN question_quality_score >= 254.86 THEN 1 ELSE 0 END) AS NewTrendSetter
```

Table RAW

AnalyzedUser	NewTrendSetter
259	26

Following the user analysis, the focus shifted to examining the questions. The objective here was to identify the most active discussions and trending topics. First, the discussion state of each question was categorized by calculating an `engagement_score`. This score assigns a double weight to answers compared to comments, reflecting the higher effort required to provide technical solutions. By analyzing the data distribution, the 90th percentile (score >= 5.0) was identified as the optimal threshold to dynamically apply the `:HotTopic` label to

nodes, while questions with zero interactions were assigned the `:IgnoredTopic` label. This statistical approach ensures that the "Trending" status is not arbitrary, but rather represents the top 10% of the most active discussions within the community.

```
MATCH (q:Question)
OPTIONAL MATCH (a:Answer)-[:ANSWERED]->(q)
OPTIONAL MATCH (c:Comment)-[:COMMENTED_ON]->(q)
WITH q,
     count(DISTINCT a) AS num_risposte,
     count(DISTINCT c) AS num_commenti
WITH q,
     (num_risposte * 2) + num_commenti AS engagement_score
RETURN
     percentileCont(engagement_score, 0.5) AS Mediana,
     percentileCont(engagement_score, 0.75) AS Top_25_Percent,
     percentileCont(engagement_score, 0.90) AS Top_10_Percent
```

Table RAW

Mediana	Top_25_Percent	Top_10_Percent
2	4	5.0

```
MATCH (q:Question)
OPTIONAL MATCH (a:Answer)-[:ANSWERED]->(q)
OPTIONAL MATCH (c:Comment)-[:COMMENTED_ON]->(q)
WITH q, count(DISTINCT a) AS num_risposte, count(DISTINCT c) AS num_commenti
WITH q, num_risposte, num_commenti, (num_risposte * 2) + num_commenti AS engagement_score
SET q.engagement_score = toFloat(engagement_score),
    q.num_risposte = num_risposte,
    q.num_commenti = num_commenti

REMOVE q:HotTopic, q:IgnoredTopic

FOREACH (_ IN CASE WHEN engagement_score >= 5.0 THEN [1] ELSE [] END | SET q:HotTopic)
FOREACH (_ IN CASE WHEN num_risposte = 0 AND num_commenti = 0 THEN [1] ELSE [] END | SET q:IgnoredTopic)

RETURN count(q) AS UpdatedQuestion
```

Table RAW

DomandeAggiornate
1589

Set 4767 properties, added 406 labels, removed 406 labels

Building upon this, we identified Trending Tags by analyzing the relationship between tags and high-engagement content. Instead of simply counting total tag usage, which would favor older, static topics, we calculated a `trend_score` exclusively for tags associated with `:HotTopic` nodes. This score balances the frequency of a tag within active discussions against the total `view_count` those discussions generate, using a logarithmic scale to normalize traffic spikes. By setting a threshold at the 90th percentile (score >= 13.01), we can distinguish between generally popular topics and the specific `:TrendingTag` nodes that are currently driving the community's most intense engagement and visibility.

```
MATCH (t:Tag)<-[:TAGGED]-(q:HotTopic)
WITH t,
     count(q) AS hot_questions_count,
     sum(q.view_count) AS total_hot_views
WITH t,
     (hot_questions_count * log10(total_hot_views + 1)) AS trend_score
RETURN
     percentileCont(trend_score, 0.5) AS Mediana,
     percentileCont(trend_score, 0.75) AS Top_25_Percent,
     percentileCont(trend_score, 0.90) AS Top_10_Percent
```

Table RAW

Mediana	Top_25_Percent	Top_10_Percent
2.0	4.529635646019073	13.01359328681814

```
MATCH (t:Tag)<-[:TAGGED]-(q:HotTopic)
WITH t,
     count(q) AS hot_questions_count,
     sum(q.view_count) AS total_hot_views
WITH t, (hot_questions_count * log10(total_hot_views + 1)) AS trend_score
SET t.trend_score = round(trend_score, 2)
REMOVE t:TrendingTag
FOREACH (_ IN CASE WHEN trend_score >= 13.01 THEN [1] ELSE [] END |
    SET t:TrendingTag
)

RETURN
    count(t) AS AnalyzedTag,
    sum(CASE WHEN trend_score >= 13.01 THEN 1 ELSE 0 END) AS NewTrendingTags
```

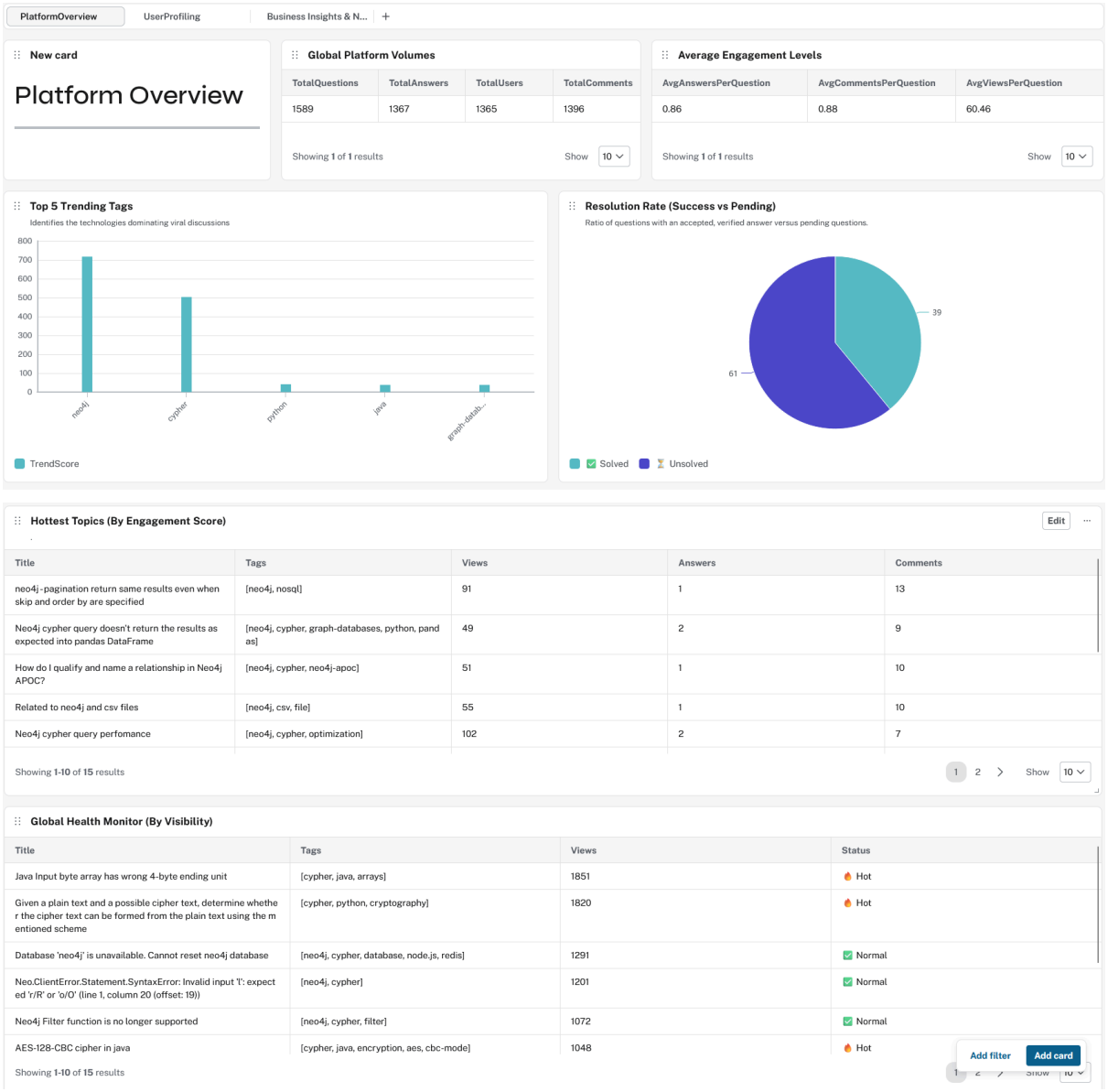
Table RAW

AnalyzedTag	NewTrendingTags
109	11

4. Dashboard Configuration

To provide a logical and comprehensive flow of information, the interface was structured into three distinct pages, each serving a specific analytical purpose. The construction of each page took place by leveraging the labelling work done previously..

The first page, dedicated to Global Platform Overview and Health, acts as the entry point. It provides a macro-level snapshot of the community's size, vitality, and content trends. It displays high-level KPIs such as top 5 Trending Tags or Resolution Question Rate. In addiction evaluates the platform's overall effectiveness using a global health monitor, which identifies highly viewed topics and categorizes them..



Below, we list the queries associated with the first page

Global Platform Volumes

Add description

Query Settings Actions

Table

```
RETURN
  count { MATCH (:Question) } AS TotalQuestions,
  count { MATCH (:Answer) } AS TotalAnswers,
  count { MATCH (:User) } AS TotalUsers,
  count { MATCH (:Comment) } AS TotalComments
```

Average Engagement Levels

Add description

Query Settings Actions

Table

```
MATCH (q:Question)
WITH count(q) AS qCount,
sum(q.view_count) AS vSum,
sum(q.answer_count) AS aSum
MATCH (c:Comment)
WITH qCount, vSum, aSum, count(c) AS cCount
RETURN
  round(toFloat(aSum) / qCount, 2) AS AvgAnswersPerQuestion,
  round(toFloat(cCount) / qCount, 2) AS AvgCommentsPerQuestion,
  round(toFloat(vSum) / qCount, 2) AS AvgViewsPerQuestion
```

Resolution Rate (Success vs Pending)

Ratio of questions with an accepted, verified answer versus pending questions.

Query Settings Pie chart

Natural language Cypher Preview

```
MATCH (q:Question)
OPTIONAL MATCH (q)<-[:ANSWERED]-(a:Answer)
WHERE a.is_accepted = true

WITH q, count(a) > 0 AS IsResolved
WITH
  CASE WHEN IsResolved THEN '✅ Solved'
  ELSE '⏳ Unsolved' END AS Status,
  count(q) AS Volume

MATCH (total:Question)
WITH Status, Volume, count(total) AS totalQ
RETURN
  Status,
  Volume,
  round((toFloat(Volume) / totalQ) * 100, 1) AS Percentage
```

Top 5 Trending Tags

Identifies the technologies dominating viral discussions

Query Settings

Bar chart

Cypher

```
MATCH (t:TrendingTag)
RETURN t.name AS Tag, t.trend_score AS TrendScore
ORDER BY TrendScore DESC
LIMIT 5
```

Hottest Topics (By Engagement Score)

.Highlights the top 15 questions by engagement score, detailing their tags, views, answers, and comments

Query Settings Actions Table

Natural language Cypher Preview

```
MATCH (q:Question)
WITH q
ORDER BY q.engagement_score DESC
LIMIT 15

MATCH (q)-[:TAGGED]->(t:Tag)
WITH q, collect(t.name) AS TagList,
     replace(replace(q.title, '&#39;', "'"), '&quot;', '"') AS CleanTitle

RETURN CleanTitle AS Title,
       TagList AS Tags,
       q.view_count AS Views,
       q.num_risposte AS Answers,
       q.num_commenti AS Comments

ORDER BY  q.engagement_score DESC
```

Global Health Monitor (By Visibility)

"Highlights the top 15 most-viewed questions, detailing their tags and status (Hot/Ignored) to reveal market gaps.

Query Settings Actions Table

Natural language Cypher Preview

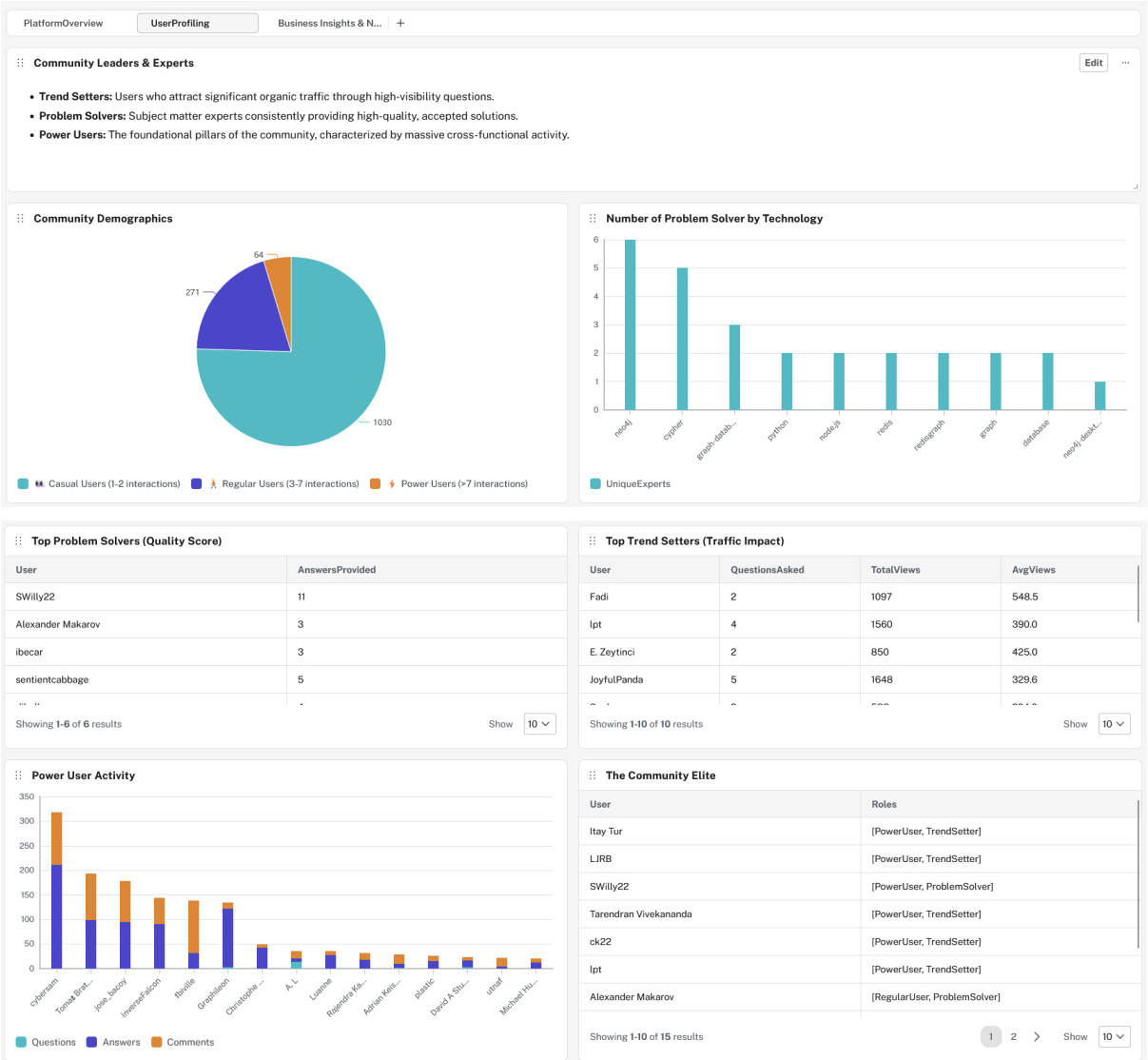
```
MATCH (q:Question)
WITH q
ORDER BY q.view_count DESC
LIMIT 15

MATCH (q)-[:TAGGED]->(t:Tag)
WITH q, collect(t.name) AS TagList,
     replace(replace(q.title, '&#39;', "'"), '&quot;', '"') AS CleanTitle

RETURN CleanTitle AS Title,
       TagList AS Tags,
       q.view_count AS Views,
       CASE
         WHEN q:HotTopic THEN '🔥 Hot'
         WHEN q:IgnoredTopic THEN '⚠️ Ignored'
         ELSE '✅ Normal'
       END AS Status
ORDER BY Views DESC
```

The second page shifts the focus from the content to the creators, diving deep into User Profiling and Community Leaders, leveraging the custom labels assigned during the data preparation phase. It starts with a broad demographic breakdown of casual, regular, and

power users, and then drills down into specific community archetypes. It highlights the top problem solvers and trend setters, ranking them by the actual quality and impact of their contributions rather than raw activity. The true value of this page lies in its ability to identify the community elite: those rare, cross-functional users who excel in both driving traffic with relevant questions and providing high-quality technical solutions.



Below, we list the queries associated with the second page

Community Demographics

Segments the user base into Power, Regular, and Casual tiers to show the total volume of users by interaction level

Query | Settings | Pie chart

Natural language | Cypher | ▷ Preview

```
MATCH (u:User)
WHERE u:PowerUser OR u:RegularUser OR u:CasualUser
RETURN CASE
         WHEN u:PowerUser THEN '⚡ Power Users (>7 interactions)'
         WHEN u:RegularUser THEN '🚶 Regular Users (3-7 interactions)'
         WHEN u:CasualUser THEN '👀 Casual Users (1-2 interactions)'
       END AS UserType,
       count(u) AS Volume
ORDER BY Volume DESC
```

Number of Problem Solver by Technology

Ranks the top 10 technologies based on the total number of unique contributing experts

Query | Settings | Bar chart

Natural language | Cypher | ▷ Preview

```
MATCH (u:User:ProblemSolver)-[:PROVIDED]->(a:Answer)-[:ANSWERED]->(q:Question)-[:TAGGED]->(t:Tag)
RETURN t.name AS Technology, count(DISTINCT u) AS UniqueExperts
ORDER BY UniqueExperts DESC
LIMIT 10
```

Top Trend Setters (Traffic Impact)

Add description

Query | Settings | Actions

Table

Natural language | Cypher | ▷ Preview

```
MATCH (u:User:TrendSetter)-[:ASKED]->(q:Question)
WITH u,
     count(q) AS QuestionsAsked,
     sum(q.view_count) AS TotalViews,
     round(avg(q.view_count), 2) AS AvgViews,
     u.question_quality_score AS score
RETURN u.display_name AS User,
       QuestionsAsked,
       TotalViews,
       AvgViews
ORDER BY score DESC
LIMIT 10
```

Power User Activity

Ranks the top 15 Power Users by total community interactions

Query | Settings | Bar chart

Natural language | Cypher | ▷ Preview

```
MATCH (u:User:PowerUser)
OPTIONAL MATCH (u)-[:ASKED]->(q:Question)
OPTIONAL MATCH (u)-[:PROVIDED]->(a:Answer)
OPTIONAL MATCH (u)-[:COMMENTED]->(c:Comment)
WITH u, count(DISTINCT q) AS Questions,
count(DISTINCT a) AS Answers, count(DISTINCT c)
AS Comments
RETURN u.display_name AS User,
       Questions,
       Answers,
       Comments
ORDER BY (Questions + Answers + Comments) DESC
LIMIT 15
```

The Community Elite

Identifies 15 users holding two or more distinct community roles

Query | Settings | Actions

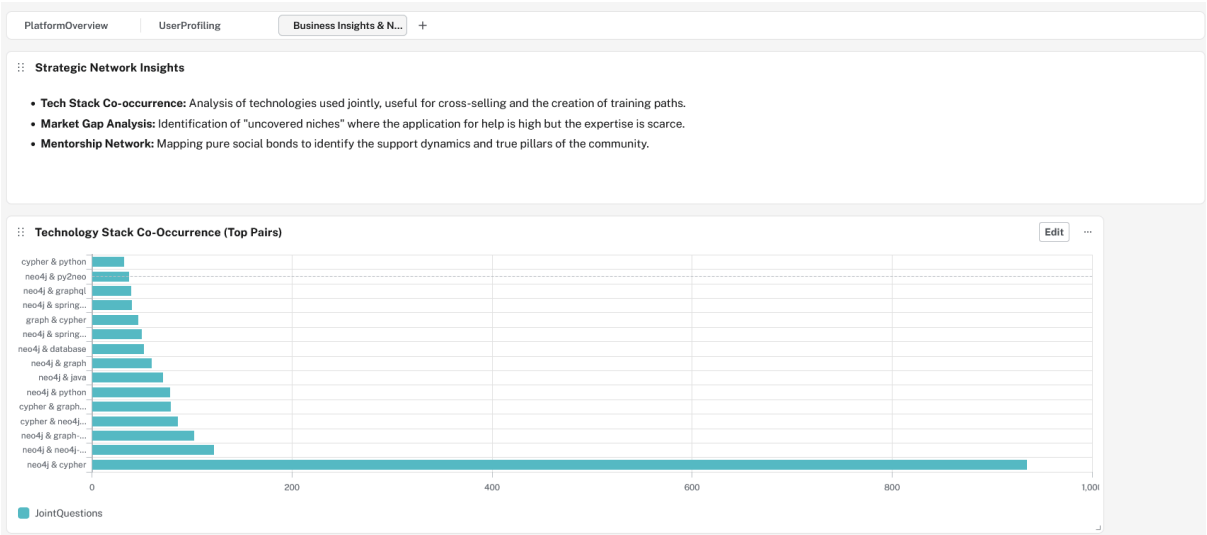
Table

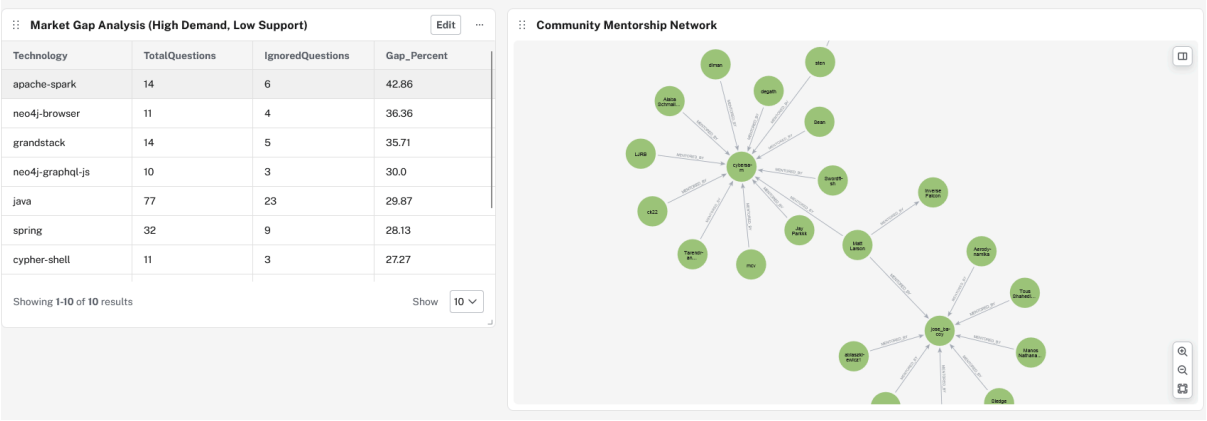
Natural language | Cypher | ▷ Preview

```
MATCH (u:User)
WITH u, [l IN labels(u) WHERE l IN
['PowerUser', 'RegularUser', 'CasualUser',
'TrendSetter', 'ProblemSolver']] AS Roles
WHERE size(Roles) >= 2

RETURN u.display_name AS User,
       Roles
ORDER BY ('PowerUser' IN Roles) DESC,
         ('RegularUser' IN Roles) DESC,
         User ASC
LIMIT 15
```

Finally, the third page explores hidden relationships and structural dynamics. It analyzes technology stack co-occurrence to reveal which tools developers actually use together in the real world, providing valuable data for cross-selling or content strategy. Furthermore, it visualizes the mentorship social network by mapping the direct support bonds between users. By leveraging virtual relationships to draw the graph, it highlights the central hubs and mentors of the community, ultimately revealing the underlying social resilience and the true human pillars of the platform. Finally, identify any market gaps: technologies where there is high demand but little support from the community.





Below, we list the queries associated with the third page

Technology Stack Co-Occurrence (Top Pairs)

Identifies the top 15 most frequent technology pairings

Query | Settings | Bar chart

Natural language | Cypher | Preview

```
MATCH (t1:Tag)<-[:TAGGED]-(q:Question)-
[:TAGGED]->(t2:Tag)
//avoid duplicate
WHERE id(t1) < id(t2)
RETURN t1.name + ' & ' + t2.name AS TechPair,
       count(q) AS JointQuestions
ORDER BY JointQuestions DESC
LIMIT 15
```

Market Gap Analysis (High Demand, Low Support)

Highlights critical market gaps by ranking the top 10 technologies with the highest percentage of unresolved or ignored questions

Query | Settings | Actions | Table

Natural language | Cypher | Preview

```
MATCH (q:Question)-[:TAGGED]->(t:Tag)

WITH t, count(q) AS TotalQuestions, sum(CASE WHEN q:IgnoredTopic THEN 1 ELSE 0 END) AS
Unresolved

WHERE TotalQuestions >= 10

WITH t, TotalQuestions, Unresolved, round((toFloat(Unresolved) / TotalQuestions) * 100, 2) AS
GapPercentage

RETURN t.name AS Technology,
       TotalQuestions,
       Unresolved AS IgnoredQuestions,
       GapPercentage AS Gap_Percent

ORDER BY GapPercentage DESC, TotalQuestions DESC
```

Community Mentorship Network

Generates a virtual graph of community mentorships, highlighting the top 30 strongest user-to-user connections based on repeated Q&A interactions

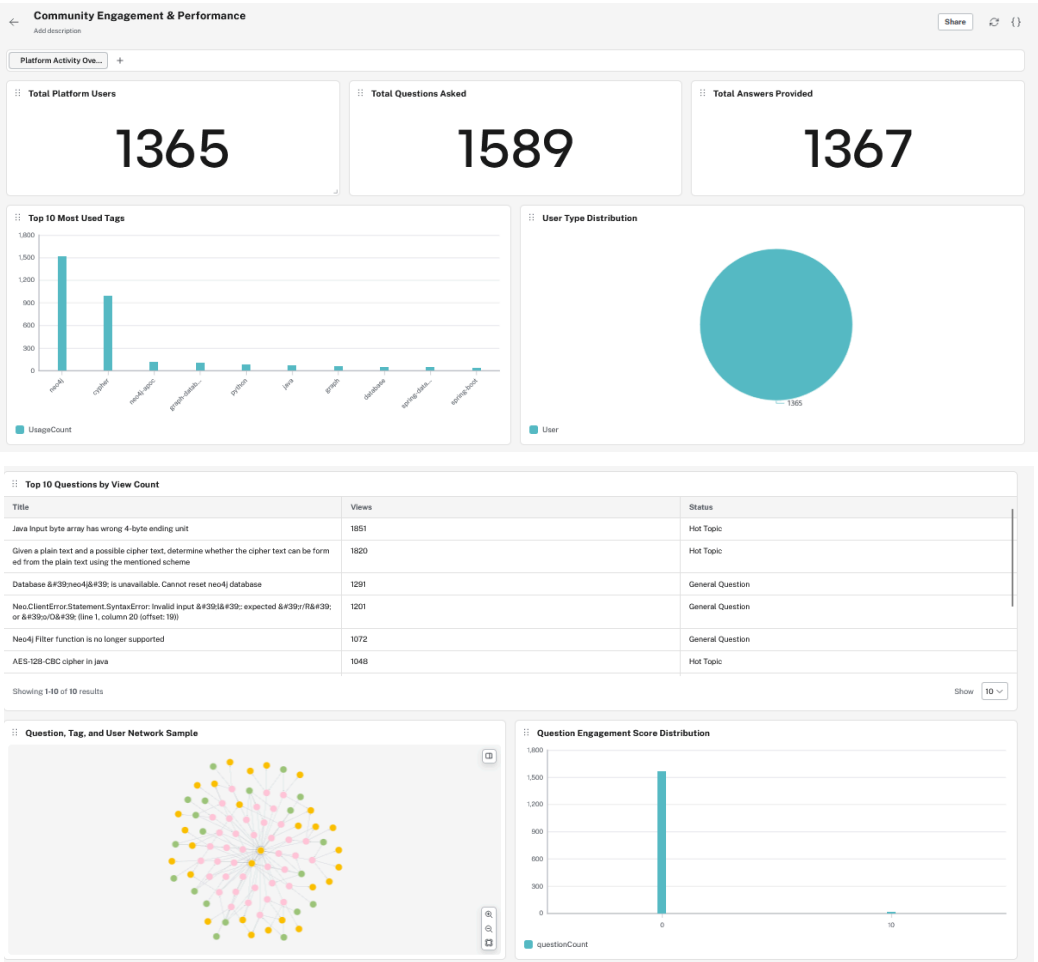
Query | Settings | Graph

Natural language | Cypher | Preview

```
MATCH (mentor:User)-[:PROVIDED]->(a:Answer)-[:ANSWERED]->(q:Question)<-[:ASKED]-(mentee:User)
WHERE mentor <> mentee
WITH mentor, mentee, count(a) AS interactions
ORDER BY interactions DESC
LIMIT 30
WITH mentor, mentee, interactions,
    apoc.create.vRelationship(mentee, 'MENTORED_BY', {weight: interactions}, mentor) AS rel
RETURN mentor, mentee, rel
```

Finally, the study explored the automated dashboard generation capabilities within Neo4j Aura. A significant technical limitation was identified concerning the NeoDash "Generate Dashboard with AI" workflow. The AI engine operates on a global scope; consequently, it is currently unable to append individual pages or components to an existing project. Each prompt execution generates an entirely new single-page dashboard. To achieve a complex multi-page architecture while leveraging AI assistance, a hybrid methodology must be adopted: the analyst must first manually define the dashboard pages and then populate them by accessing individual cards. Within each component, the "Generate Query with Natural Language" function can be utilized to build specific visualizations, bypassing the limitations of the global dashboard generator. The initial attempt to generate a dashboard using the prompt below yielded suboptimal results, as illustrated in figure.

"Generate a dashboard page for platform overview. Include big number KPIs for total Users, Questions, and Answers. Add a Bar Chart showing the most used Tags. Create a Table for 'Top 10 Questions' ordered by view_count, including a column for the 'Status' label (HotTopic or IgnoredTopic) to monitor platform health."

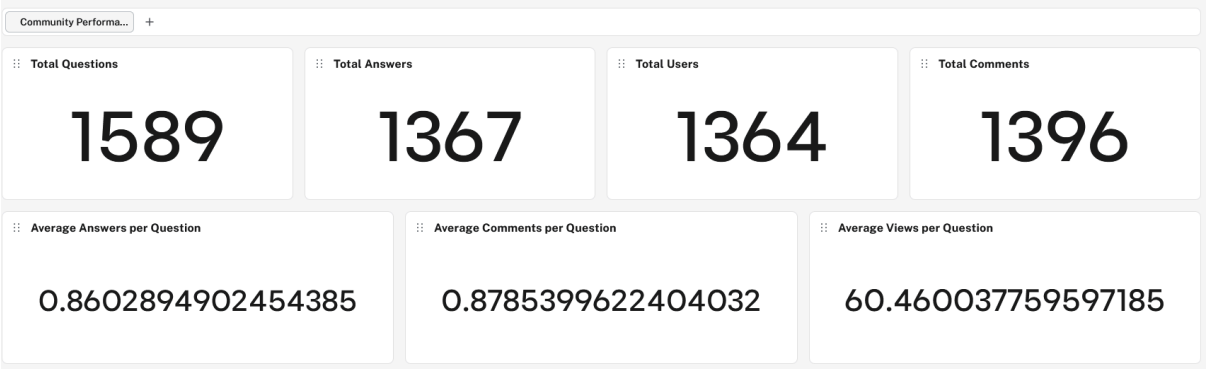


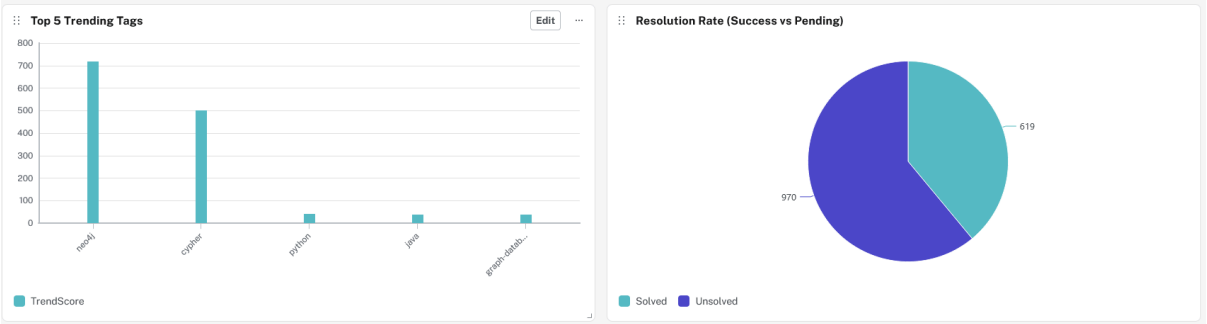
Subsequently, through a more refined prompt engineering process, the AI-generated dashboard achieved full parity with the manually constructed version previously developed.

"Create a dashboard page titled 'Platform Overview' following these specific instructions:

1. *Header: Add a Markdown card at the top-left titled 'Platform Overview' with a clean, professional description of the dashboard's purpose.*
2. *KPIs (Big Numbers):*
   - *Create a chart titled 'Global Platform Volumes' showing four metrics: total Questions, total Answers, total Users, and total Comments.*
   - *Create a chart titled 'Average Engagement Levels' calculating the average number of answers per question, comments per question, and views per question.*
3. *Visual Charts:*
   - *Bar Chart: Titled 'Top 5 Trending Tags'. It must use the `TrendScore` property of the Tag nodes, ordered descending, showing the top 5 results.*
   - *Pie Chart: Titled 'Resolution Rate (Success vs Pending)'. It should show the ratio between questions with a verified/accepted answer ('Solved') and questions without one ('Unsolved').*
4. *Detailed Tables:*
   - *Table 1: Titled 'Hottest Topics (By Engagement Score)'. Show the top 15 questions by `engagement_score`. Columns: Title (cleaned from HTML entities), Tags, Views, Answers (use `num_risposte`), and Comments (use `num_commenti`).*
   - *Table 2: Titled 'Global Health Monitor (By Visibility)'. Show the top 15 questions by `view_count`. Columns: Title, Tags, Views, and a 'Status' column. The Status should map the labels: `HotTopic` to '🔥 Hot', `IgnoredTopic` to '⚠️ Ignored', and everything else to '✅ Normal'.*

Ensure the layout places the KPIs at the top and the tables at the bottom, matching a professional BI report structure."





Hottest Topics (By Engagement Score)

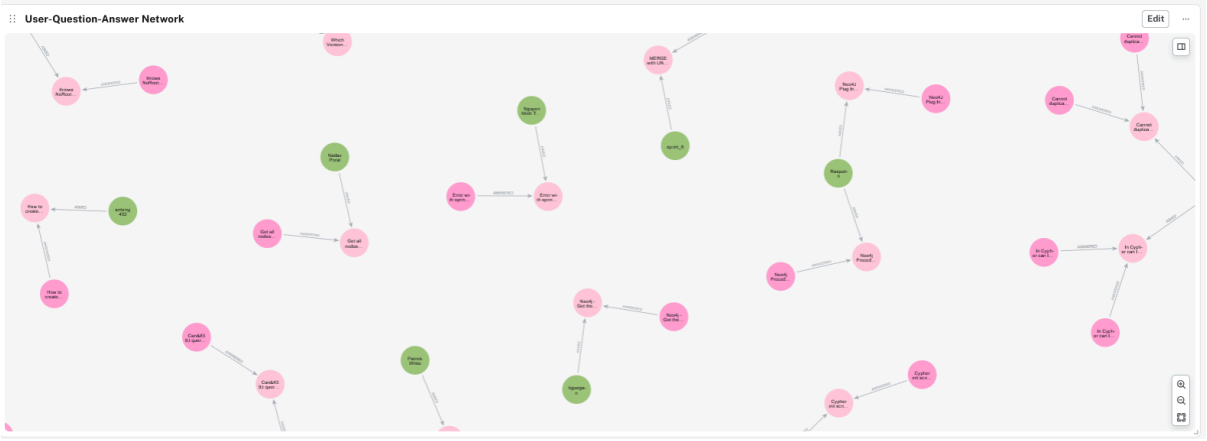
Title	Tags	Views	Answers	Comments
neo4j -pagination return same results even when skip and order by are specified	[neo4j, nosql]	91	1	13
Neo4j cypher query doesn't return the results as expected into pandas DataFrame	[neo4j, cypher, graph-databases, python, pandas]	49	2	9
How do I qualify and name a relationship in Neo4j APOC?	[neo4j, cypher, neo4j-apoc]	51	1	10
Related to neo4j and csv files	[neo4j, csv, file]	55	1	10
Neo4j cypher query performance	[neo4j, cypher, optimization]	102	2	7
Swapping elements in a list based on their match in a dictionary	[cypher, python, cryptography, dictionary, alphabet]	58	3	5
Connection Unavailable: Trying to connect locally hosted Neo4j from docker app	[neo4j, docker, py2neo, fastapi]	280	1	9
Kubernetes pod FileLockException: Lock file has been locked by another process - neo4j 4.2	[neo4j, java, docker, kubernetes]	42	1	9
AND clause for matching 3 conditions in NEO4j	[neo4j]	33	1	9

Showing 1-10 of 15 results 1 2 Show 10

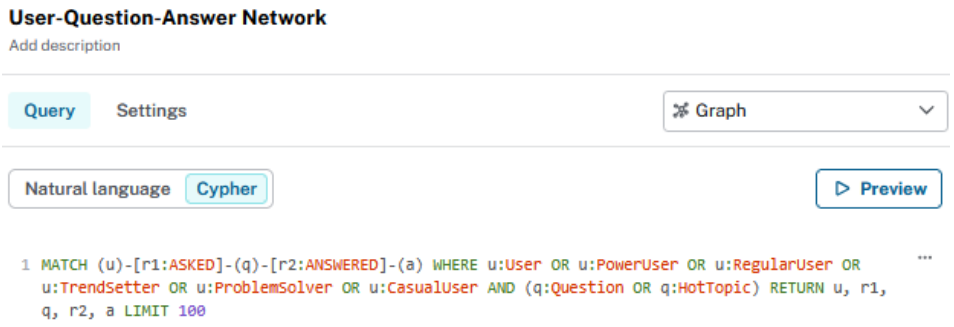
Global Health Monitor (By Visibility)

Title	Tags	Views	Status
Java Input byte array has wrong 4-byte ending unit	[cypher, java, arrays]	1851	Hot
Given a plain text and a possible cipher text, determine whether the cipher text can be formed from the plain text using the mentioned scheme	[cypher, python, cryptography]	1820	Hot
Database 'neo4j' is unavailable. Cannot reset neo4j database	[neo4j, cypher, database, node.js, redis]	1291	Normal
Neo.ClientError.Statement.SyntaxError: Invalid input '|': expected 'r/R' or 'o/O' (line 1, column 20 (offset: 19))	[neo4j, cypher]	1201	Normal
Neo4j Filter function is no longer supported	[neo4j, cypher, filter]	1072	Normal
AES-128-CBC cipher in java	[cypher, java, encryption, aes, cbc-mode]	1048	Hot
Cypher: what is the use of YIELD?	[neo4j, cypher]	929	Normal
Cypher commands not being recognised	[neo4j, cypher]	905	Normal
How can I use the `with` statement with `call.apoc.do.when` in Neo4j?	[neo4j, cypher]	809	Normal
Efficient way to import multiple csv's in neo4j	[neo4j, cypher, load-csv]	698	Hot

Showing 1-10 of 15 results 1 2 Show 10



As observed in both instances, the AI introduced an unsolicited query that maps out how different user personas interact with trending or standard questions, identifying the specific answers connected to those interactions. The generated Cypher code is presented below:



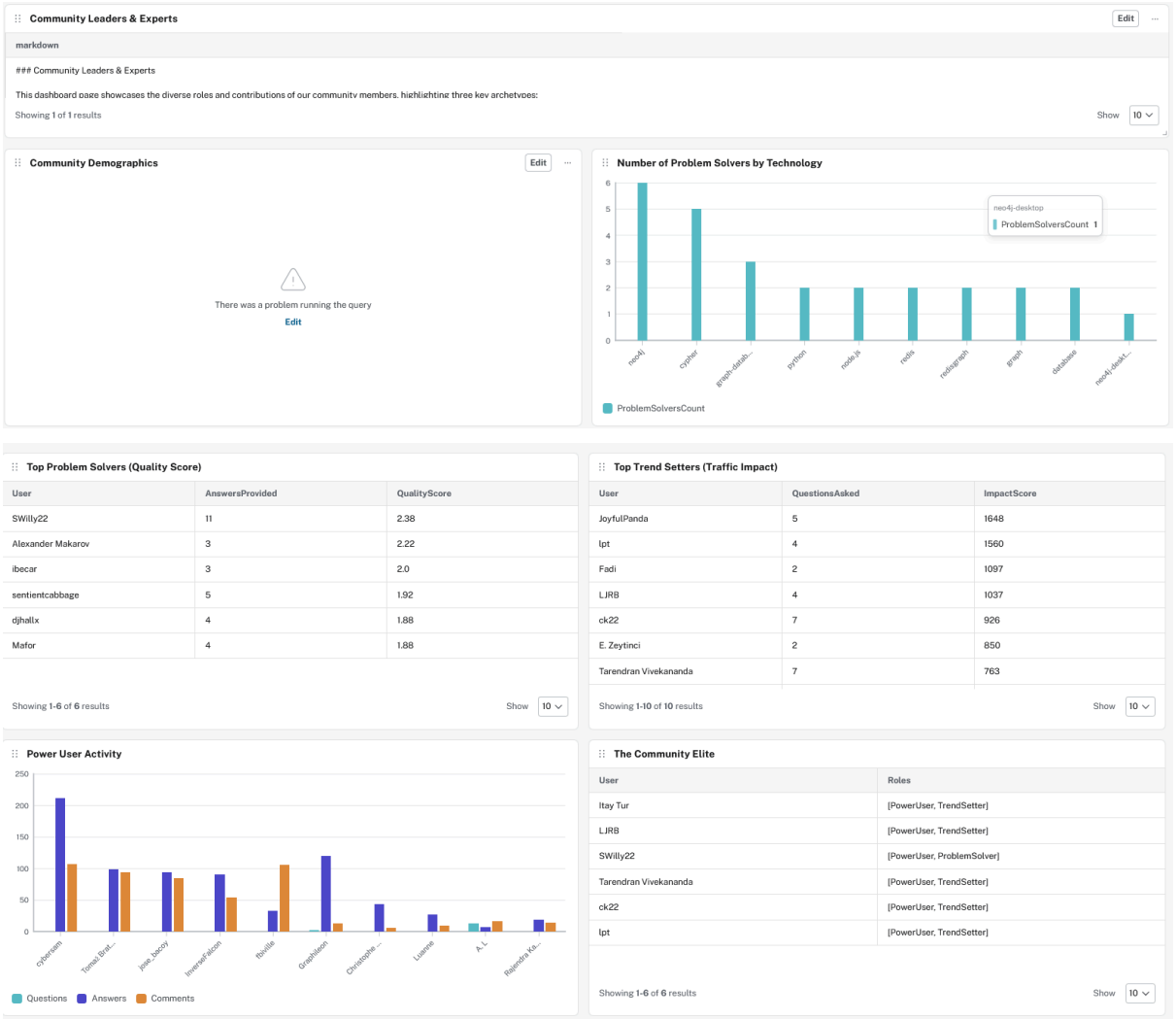
The following prompt was utilized for the generation of the second page:

"Generate a dashboard page titled 'User Profiling' with a professional layout and 6 components. Follow these exact instructions:

1. *Header: Add a Markdown card at the top titled 'Community Leaders & Experts' describing the three main archetypes: Trend Setters (traffic generators), Problem Solvers (quality experts), and Power Users (highly active pillars).*
2. *Demographics (Pie Chart): Create a chart titled 'Community Demographics'. It must show the distribution of the following labels: `CasualUser`, `RegularUser`, and `PowerUser`.*
3. *Expertise (Bar Chart): Create a chart titled 'Number of Problem Solver by Technology'. It should count unique nodes with the label `ProblemSolver` associated with each `Tag` through the `[:TAGGED]` and `[:PROVIDED]` relationship path.*
4. *Problem Solvers (Table): Create a table titled 'Top Problem Solvers (Quality Score)'. Show the top 10 users with the label `ProblemSolver`, ordered by their `answer_quality_score` property. Columns: User (display_name), AnswersProvided (use `num_risposte`), and QualityScore (use `answer_quality_score`).*
5. *Trend Setters (Table): Create a table titled 'Top Trend Setters (Traffic Impact)'. Show the top 10 users with the label `TrendSetter`. Columns: User (display_name), QuestionsAsked (use `num_domande`), and ImpactScore (use `question_quality_score` or calculate sum of `view_count` from their questions).*
6. *Activity Matrix (Stacked Bar Chart): Create a chart titled 'Power User Activity'. For the top 10 `PowerUser` nodes, show a stacked bar representing their volume of Questions, Answers, and Comments. Use the pre-calculated properties `num_domande`, `num_risposte`, and `num_commenti`.*
7. *The Elite (Table): Create a table titled 'The Community Elite'. Filter for users that have more than one label among `PowerUser`, `ProblemSolver`, and `TrendSetter`. Show the User (display_name) and a list of their active Roles (labels).*

Ensure the layout is clean, placing the header at the top, the demographics/expertise in the middle row, and the detailed tables/activity chart in the bottom rows."

The generation yielded the following result:



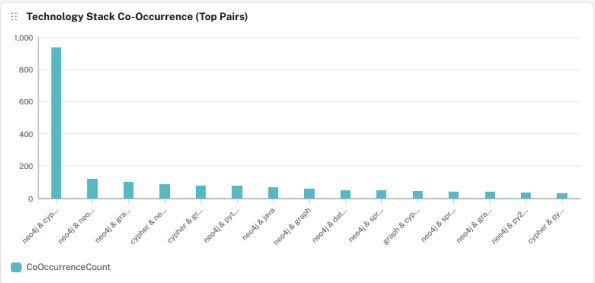
As observed, the output largely aligns with the manually developed dashboard. The primary discrepancies lie in the classification of "TrendSetters"—which the AI categorized by total view counts rather than the Question Quality Score—and the technical failure of the visualization located in the top-left quadrant. During the generation of Page 2, a significant syntactic limitation of the integrated Large Language Model (LLM) became apparent. Specifically, the AI produced a Cypher query with syntax errors, triggering a "variable not defined" exception. This outcome underscores the critical necessity of a "human-in-the-loop" approach to validate the logic and variable scope within complex, automatically generated queries.

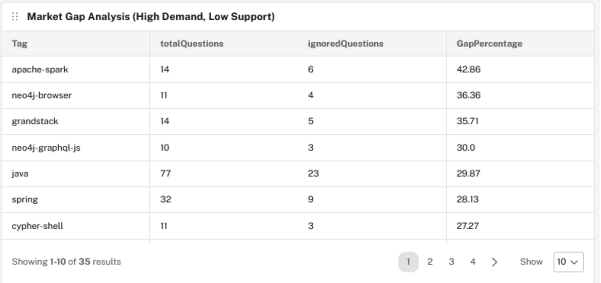
For the third page, the following prompt was utilized, yielding the results depicted in figure

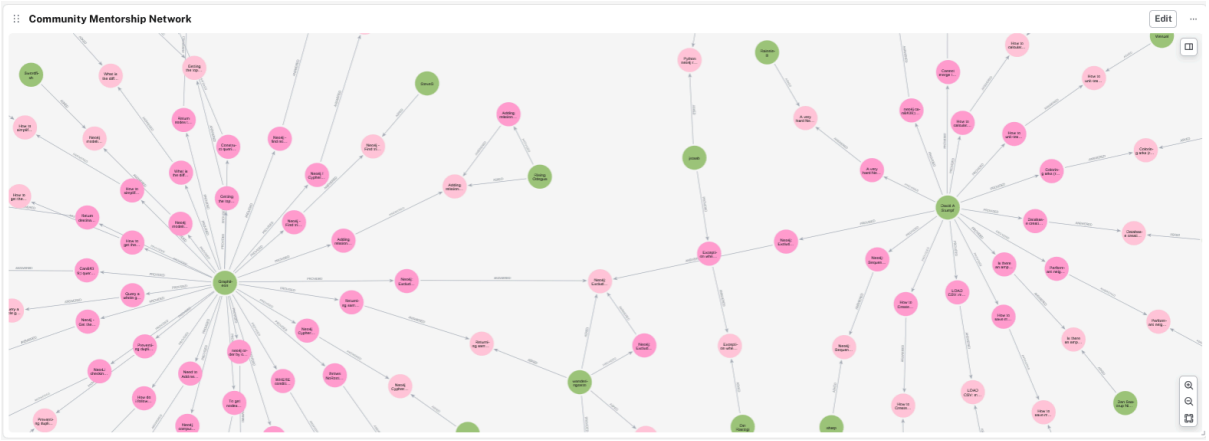
"Generate a dashboard page titled 'Business Insights & Network Analysis' with a focus on graph-driven intelligence. Follow these exact instructions:

1. *Header (Markdown): Add a Markdown card at the top titled 'Strategic Network Insights'. It should describe the three key analysis areas: 'Tech Stack Co-occurrence' (identifying tools used together), 'Market Gap Analysis' (finding unanswered niche topics), and 'Mentorship Network' (mapping human support dynamics).*
2. *Co-Occurrence (Bar Chart): Create a chart titled 'Technology Stack Co-Occurrence (Top Pairs)'. This chart must find pairs of `Tag` nodes associated with the same `Question`. Use the pattern `(t1:Tag)<-[:TAGGED]-(q:Question)-[:TAGGED]->(t2:Tag)` where `id(t1) < id(t2)`. Order by the count of shared questions and show the top 15 pairs.*
3. *Market Gap (Table): Create a table titled 'Market Gap Analysis (High Demand, Low Support)'. It must analyze `Tag` nodes. For each tag, show the total number of questions, the number of questions with the label `IgnoredTopic`, and calculate the 'Gap Percentage' (Ignored/Total). Filter for tags with at least 10 total questions and order by Gap Percentage descending.*
4. *Mentorship Network (Graph View): Create a network visualization titled 'Community Mentorship Network'. It must map the relationship between a `User` who asked a question and a `User` who provided an answer. The query should return the `User` nodes and a virtual relationship between them (representing mentorship) calculated based on the number of help interactions. Ensure the output is a Graph visualization, showing nodes with their `display_name` property.*

Ensure the layout places the Markdown header at the top, the bar chart and table in the middle row, and the mentorship graph at the bottom as a large, interactive visualization."







In this instance as well, the AI successfully generated a dashboard consistent with the manually developed version. The primary distinction, as observed, lies in the mentorship graph: while the handcrafted query focuses solely on the end-users, the AI-generated version traverses all intermediate nodes to derive the mentorship relationship.

In conclusion, although currently restricted to single-page generation, automated dashboard creation remains a functional and highly effective tool for streamlining Business Intelligence workflows. As demonstrated, successful implementation necessitates both a preliminary Data Enrichment phase to facilitate the generation process and a robust technical understanding of the database to formulate precise and comprehensive prompts.

5. Results and Insight

From the analysis of the data gathered from the graph, we can get a rather clear representation not only about the condition of the platform itself but also about its behavioral dynamics. This process begins from operational volume through quality of human resources and finally comes down to strategic relations.

1. Health and Resolution Analysis (Platform Overview)

From the first sheet of the dashboard we can conclude that the community is highly active in regards to involvement, with almost equal amount of total users (1365) and number of asked questions (1589). But from the point of view of Business Intelligence, the most interesting information can be received based on the Resolution Rate indicator. Now there are around 39% of questions resolved with an accepted answer, whereas 61% are left unanswered until now. That means that although there is a large number of interactions (over 1400 answers), the relevance of their contents does not always match the standards set by the askers.

The analysis of the average engagement also supports this perspective: while the platform enjoys great organic visibility, with about 60 views for each question, the average number of answers per question (0.86) is relatively small, which means that there are many questions that get insufficient attention. According to the Global Health Monitor, Hot Topics get thousands of views; however, without turning them into "Solved" questions, the platform runs the risk of becoming a collection of unsolved issues. In a more strategic sense, this situation shows the importance of involving experts in discussions on the hottest topics in order to increase the share of answers that were accepted.

2. User Segmentation and Valuation of Human Capital (User Profiling)

Moving our attention to the next page, it is possible to visualize the effectiveness of the community using the Pareto principle through an analysis of the user profiles. From the demographic spread, it can be seen that the operational influence is mainly held by 64 users classified as "Power Users" and who represent less than 5% of the total users.

The most revolutionary point in our analysis is the separation of quantity from quality. Thanks to the Answer Quality Score, we have found people like “SWilly22,” who, even though they are not among those with the largest number of activities on the site, still show outstanding technical knowledge and are truly the site’s “Problem Solvers.” On the other hand, there are Trend Setters like “Fadi” who bring in hundreds of viewers by means of interesting questions

which grab the attention of the outside world. By combining these roles in the table of Community Elite, we can find the “Unicorns,” individuals with both the ability to draw attention and the necessary know-how to address any issues. In terms of business, the main task is to find and retain these individuals, because they are the natural leaders for each tag category.

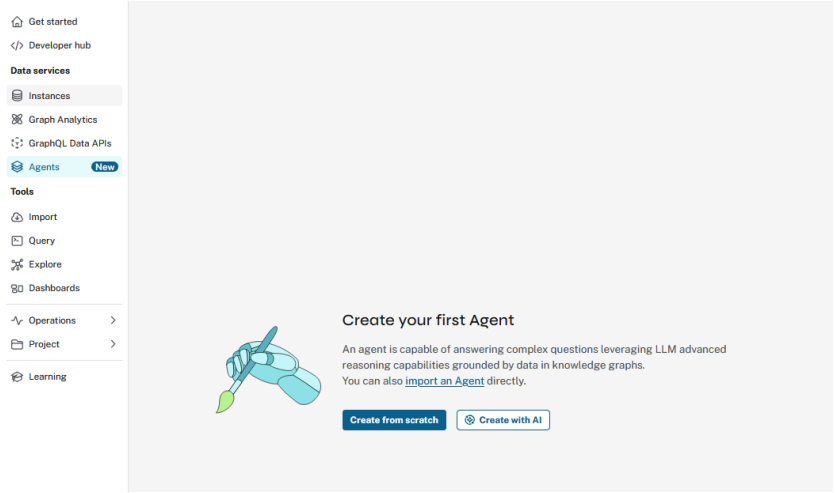
3. Strategic Insights Summary (Network & Business)

The third page analysis reveals critical strategic opportunities for optimizing both technical development and community engagement. The "Technology Stack Co-Occurrence" is overwhelmingly dominated by the Neo4j and Cypher pairing, confirming the core technical focus of the current user base. However, the "Market Gap Analysis" in the same file highlights a significant business intelligence opportunity: technologies such as `apache-spark` and `neo4j-browser` show alarmingly high gap percentages of 42.86% and 36.36%, respectively. This indicates a high demand for support that is currently unmet.. The "Community Mentorship Network" visualizes a model where specific individuals act as primary knowledge anchors for numerous queries. This represents a "knowledge silo" risk; a strategic decision would be to implement an incentive program to cultivate more mentors, thereby decentralizing the network and ensuring long-term community resilience.

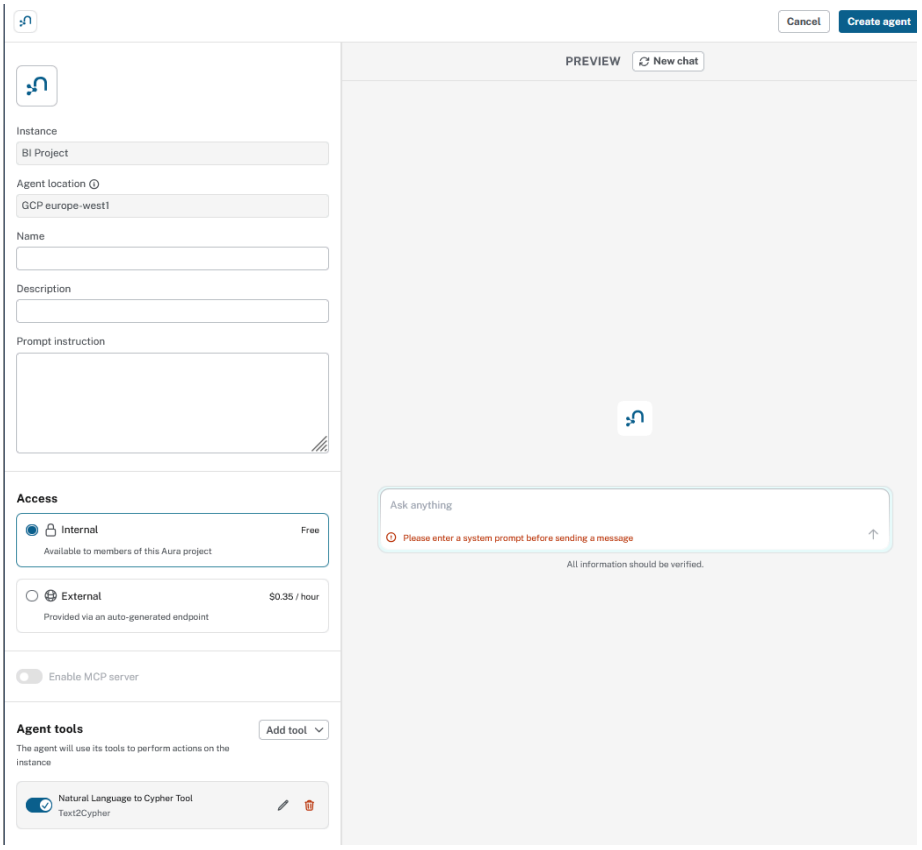
In conclusion, the platform appears to be going through a period of robust organic growth, characterized by high levels of activity volume and visibility. However, it needs better management of the expertise. In the future, the platform will need to ensure the closure of the Market Gaps by leveraging the power of elite problem solvers. It can do so through rewards. With the advent of volume analysis, profiling analysis, and network analysis, it will become feasible for the platform to manage its ecosystem sustainably and technologically.

6. AI Agent Implementation

To conclude the project, the Agents functionality within the Neo4j Aura workspace was explored.



While the platform allows for agent creation via both natural language and manual configuration, this analysis focused on the latter.



After establishing the Agent’s name and description, it was initialized through a specific system prompt:

"You are an expert data analyst for a Q&A community platform. You have access to a Neo4j graph with the following schema:

Nodes: User, Question, Answer, Tag.

Relationships: (User)-[:ASKED]->(Question), (User)-[:PROVIDED]->(Answer), (Answer)-[:ANSWERED]->(Question), (Question)-[:TAGGED]->(Tag).

You must leverage custom properties calculated during the ETL phase:

For Users: use 'answer_quality_score' and 'question_quality_score' to identify experts.

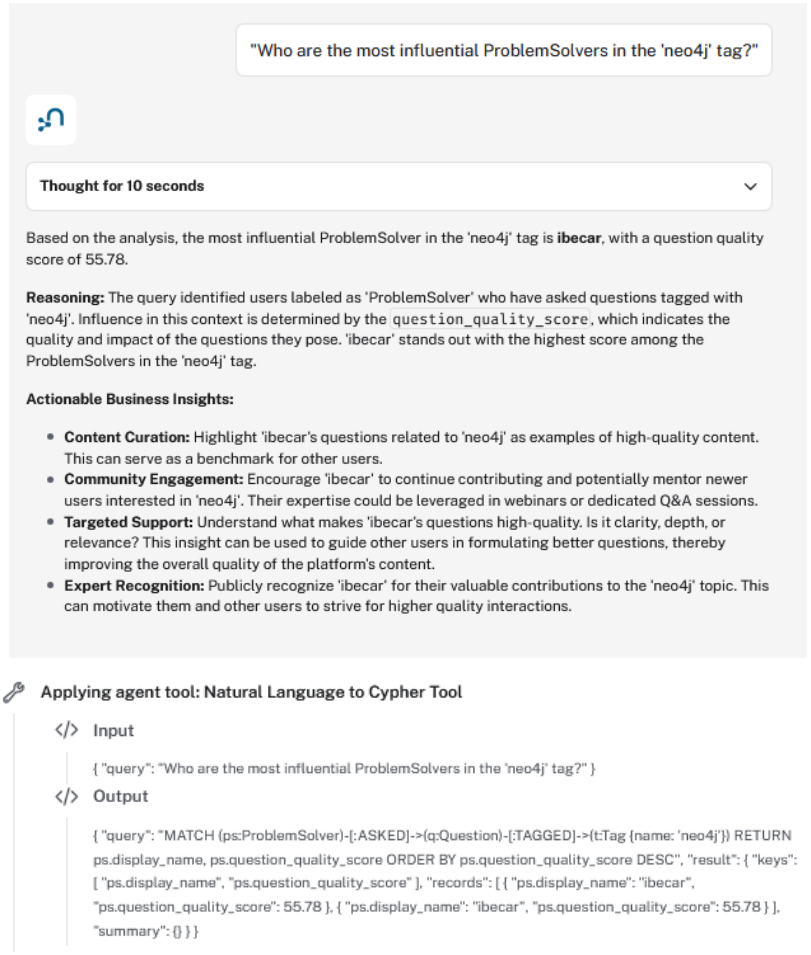
For Questions: use 'view_count' and 'engagement_score'.

Use behavioral labels: PowerUser, RegularUser, CasualUser, ProblemSolver, TrendSetter, HotTopic, IgnoredTopic.

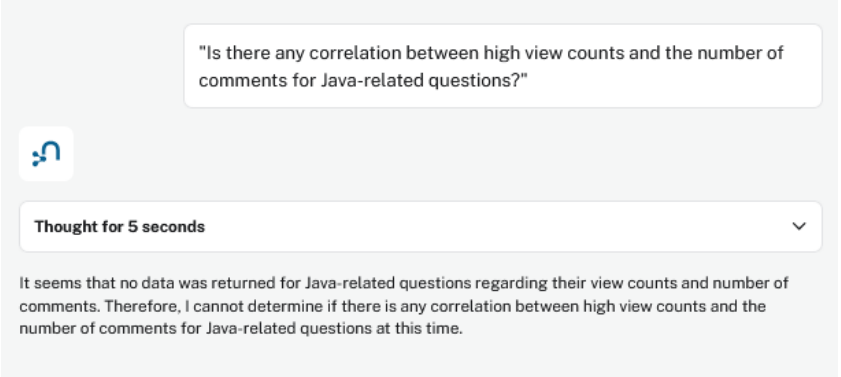
When asked a question, use the 'Natural Language to Cypher Tool' to query the graph. Explain your reasoning and provide actionable business insights based on the data retrieved."

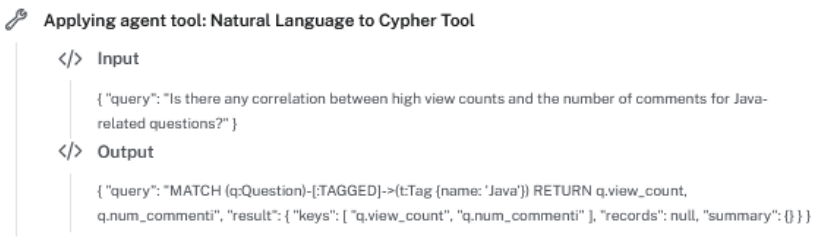
This configuration enabled a detailed evaluation of the Agent's reasoning process, allowing for a rigorous assessment of both the correctness and the quality of the generated answers. Finally, several experiments were conducted to validate the overall effectiveness and reliability of the Agent's responses.

In the initial test, the Agent was tasked with identifying the most influential users associated with "neo4j" technology. The system successfully identified the user ibecar as the primary influencer, predicated on a *question_quality_score* of 55.78. Technical analysis revealed that the Natural Language to Cypher tool accurately mapped the request by filtering ProblemSolver nodes linked to the specific tag.

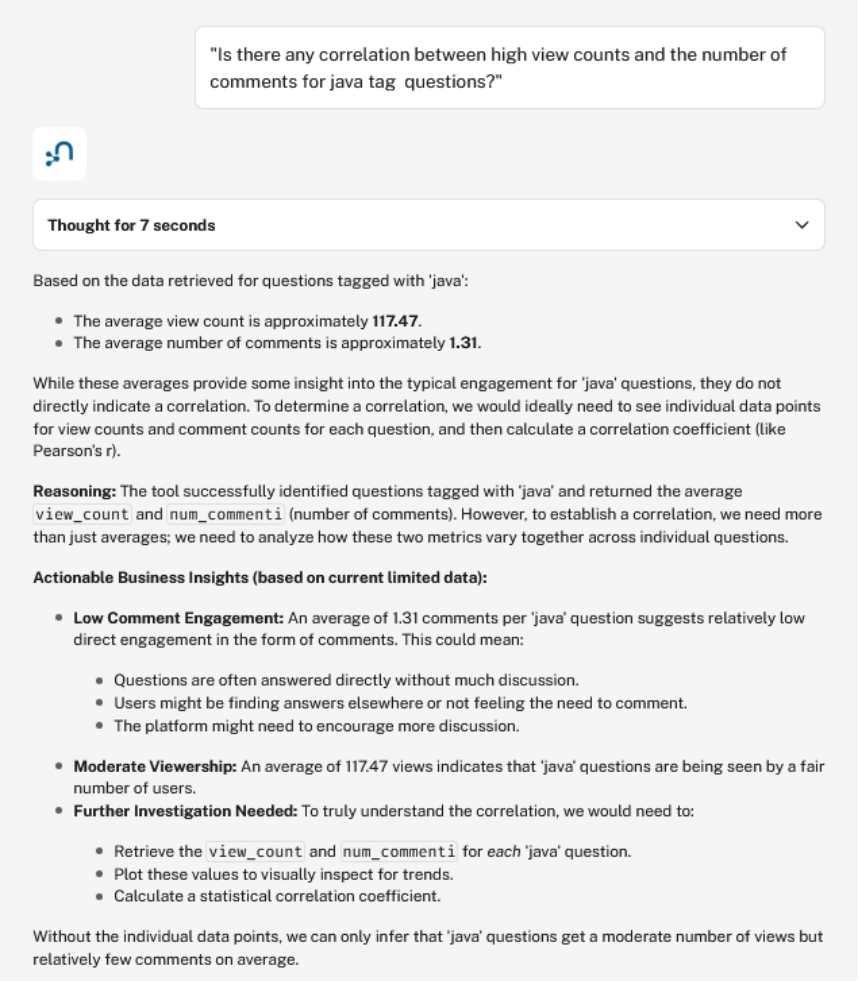


A crucial phase of the evaluation involved executing a correlation request for questions related to "Java," which highlighted a key technical limitation. The Agent initially failed to return any data because it performed a search for the exact string "Java" (case-sensitive). Since the database stores tags exclusively in lowercase, the query resulted in a null return. This finding demonstrates that the Agent is susceptible to case-sensitivity errors if not explicitly configured to normalize input terms or utilize case-insensitive search parameters.

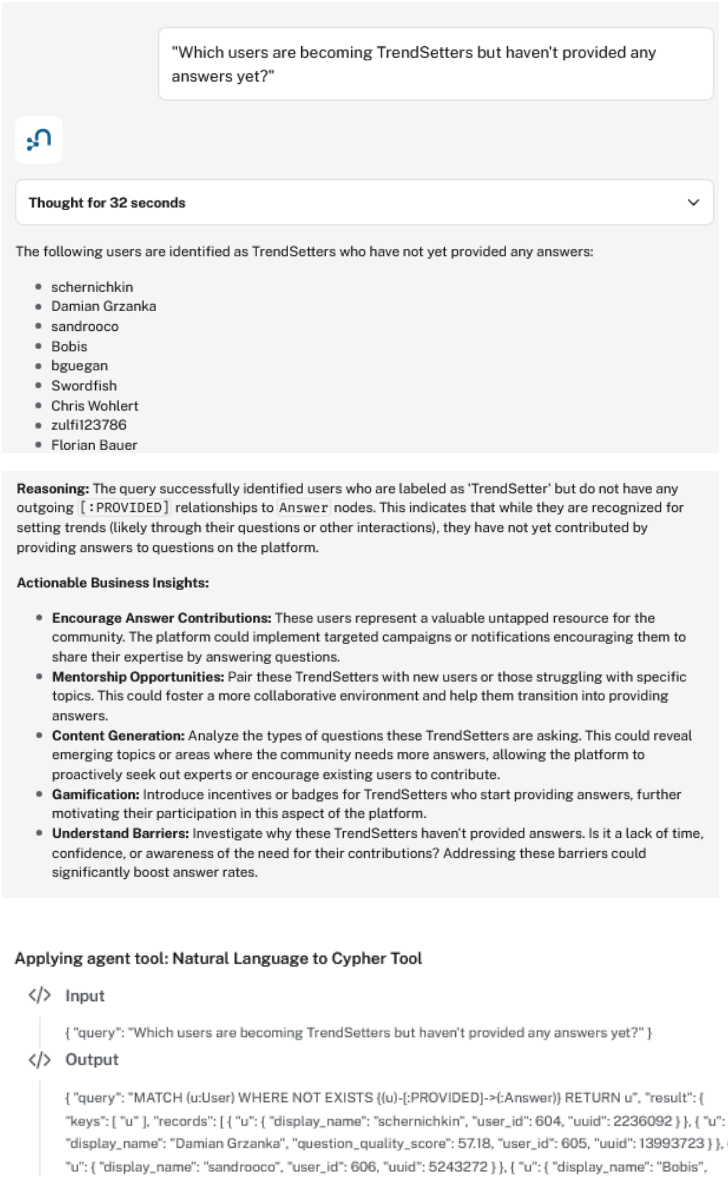




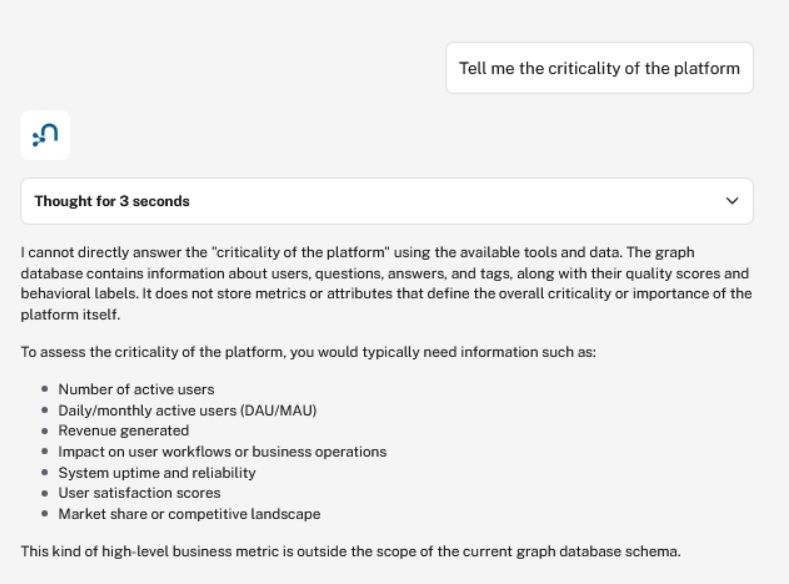
Upon repeating the query with the lowercase tag ("java"), the Agent successfully executed the analysis. Showcasing its reasoning capabilities, the system calculated the average number of views (approximately 117.47) and comments (approximately 1.31). Furthermore, the Agent demonstrated a high degree of analytical rigor by critically evaluating its own output. It explicitly clarified that simple averages are insufficient to establish a scientific correlation, correctly suggesting the necessity of computing a formal statistical metric, such as the Pearson correlation coefficient (Pearson's *r*), to properly validate the relationship.



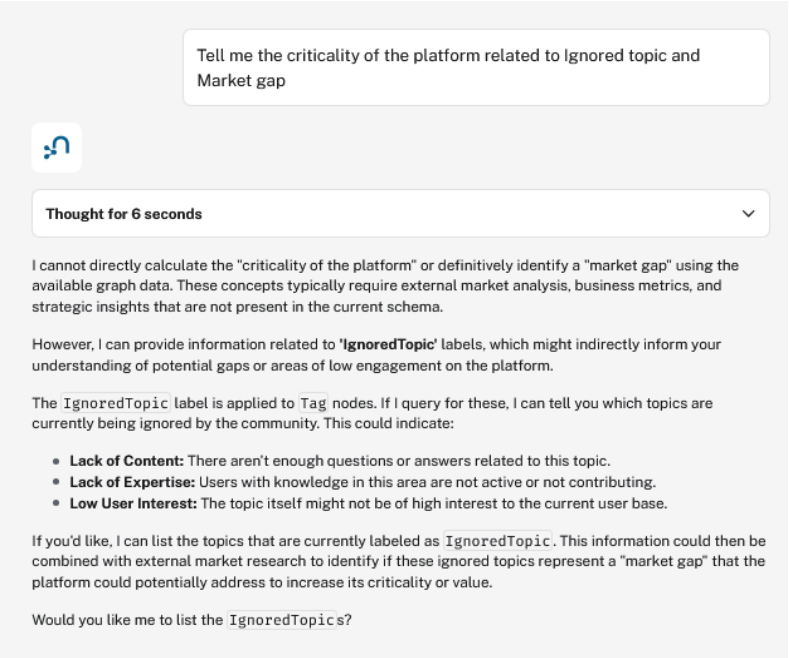
The fourth test focused on identifying users classified as "TrendSetters" who had not yet provided any responses. The analysis of this scenario revealed a critical limitation regarding the misalignment between the Agent's intent and its syntactic execution. While the Agent's reasoning process correctly identified the need to filter by the "TrendSetter" label, the generated Cypher query omitted this specific constraint. As a result, the query produced an excessively broad dataset encompassing all users without answers, regardless of their classification. This discrepancy highlights that, in a corporate environment, AI Agent outputs necessitate rigorous human validation to prevent business decisions from being predicated on technically inaccurate datasets.



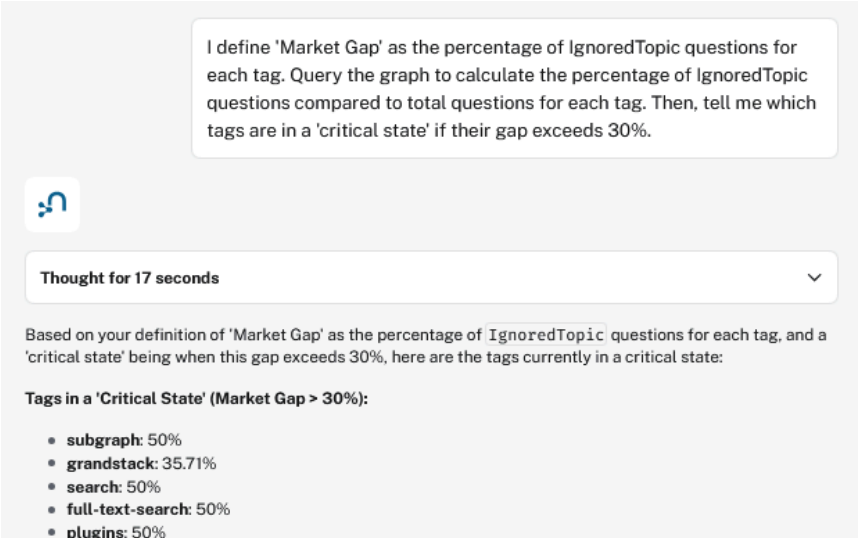
The fifth test involved a general inquiry regarding "platform criticality," which resulted in a functional failure. The Agent declined to provide a response, citing that the database schema—consisting of users, questions, and tags—does not contain the intrinsic metadata required to define "criticality" or strategic importance. It suggested that external data, such as revenue or system uptime, would be necessary for such an assessment. While this outcome demonstrated high analytical rigor and a notable absence of hallucinations—as the AI correctly avoided fabricating subjective metrics unsupported by the data—it also highlighted a lack of proactivity. The Agent was unable to autonomously propose using existing metrics, such as ignored tags, to provide an approximate or proxy analysis.



In the sixth test, the user attempted to guide the Agent by explicitly linking platform "criticality" to the concepts of IgnoredTopic and "market gaps." The outcome was a partial success, categorized as primarily informative. Analysis revealed that the Agent successfully recognized the relevance of the IgnoredTopic labels, correctly articulating that these indicate a deficit in content or specific community expertise. However, the Agent’s response was limited to a theoretical explanation; while it proposed listing the relevant tags, it failed to perform any autonomous analytical calculations. Despite this limitation, the test highlighted the system's strong semantic interpretation capabilities, as the AI effectively bridged the gap between abstract business terminology and specific technical data structures.



In the final phase of testing, a precise mathematical definition was provided to the Agent: "Market Gap" was defined as the percentage of IgnoredTopic questions for each tag, with a criticality threshold established for any value exceeding 30%. This intervention resulted in a total success. Once provided with these clear parameters, the Agent required 17 seconds to process and execute a complex Cypher query. It accurately calculated the gap percentages for each technology, identifying critical states for specific tags such as subgraph (50%), apache-spark (42.86%), and neo4j-browser (36.36%).



Reasoning: The query successfully identified tags where the proportion of questions classified as `IgnoredTopic` is significantly high (over 30%). This indicates that for these specific topics, a large portion of the related questions are not receiving adequate attention or engagement on the platform.

Actionable Business Insights:

- **Content Gap Analysis:** These tags represent significant "market gaps" within your platform, as defined by your criteria. The high percentage of `IgnoredTopic` questions suggests that users are asking about these topics, but the community isn't effectively addressing them.
- **Targeted Community Building:** Focus efforts on recruiting experts or encouraging existing users with knowledge in these critical areas to contribute answers.
- **Promote Under-Answered Questions:** Highlight questions related to these tags to increase their visibility and encourage community members to provide solutions.
- **Review Tagging Strategy:** For tags with 100% `IgnoredTopic` questions, investigate if these are niche topics with very few questions overall, or if they represent a complete lack of engagement. It might also indicate that the `IgnoredTopic` label is being applied too broadly or that the questions themselves are of low quality.
- **Educational Initiatives:** Consider creating tutorials, documentation, or hosting Q&A sessions around these critical topics to build a knowledge base and stimulate discussion.
- **Platform Criticality:** A high number of tags in a "critical state" (as defined by your market gap) suggests that a significant portion of user inquiries are going unanswered or unengaged. This could lead to user dissatisfaction, reduced platform utility, and users seeking answers elsewhere, potentially impacting the overall health and growth of the platform. Addressing these gaps is crucial for maintaining and improving the platform's value proposition.

Applying agent tool: Natural Language to Cypher Tool

Input

{ "query": "\nFor each Tag, calculate the percentage of questions that are also labeled as 'IgnoredTopic' out of the total questions for that tag.\nReturn the tag name, the total number of questions, the number of IgnoredTopic questions, and the calculated percentage.\nThen, identify and return only those tags where this percentage exceeds 30%, along with their calculated percentage, indicating they are in a 'critical state'.\n" }

Output

{ "query": "MATCH (t:Tag)<-[:TAGGED]-(q:Question) WITH t, count(q) AS totalQuestions MATCH (t)<-[:TAGGED]-(iq:IgnoredTopic) WITH t, totalQuestions, count(iq) AS ignoredQuestions WITH t.name AS tagName, totalQuestions, ignoredQuestions, (toFloat(ignoredQuestions) / totalQuestions) * 100 AS percentage WHERE percentage > 30 RETURN tagName, percentage", "result": { "keys": ["tagName", "percentage"], "records": [{ "percentage": 50, "tagName": "subgraph" }, { "percentage":

The analysis of the insights generated by the Agent revealed three distinct analytical behaviors, each presenting specific operational advantages and risks. In the test focused on identifying community leaders—such as the user "ibecar"—the Agent demonstrated a robust capability to translate technical performance scores into actionable personnel management strategies. These insights proved to be both relevant and growth-oriented, specifically by suggesting that recognized experts be transitioned into mentorship roles through public acknowledgement. However, a notable limitation was the tendency toward standardized proposals, such as generic recommendations for webinars or Q&A sessions, indicating that the AI occasionally applies broad business templates to highly specific data contexts.

The evaluation regarding TrendSetter identification serves as a critical example of how technical oversights can compromise an entire business strategy. Due to a syntax omission in the generated Cypher code—specifically the failure to filter for the ":TrendSetter" label—the Agent produced action-oriented suggestions based on a dataset that was far too broad and technically irrelevant. From a strategic perspective, implementing gamification campaigns or barrier-to-entry surveys for users who are not actual influencers would lead to a significant waste of corporate resources and the failure of marketing objectives. This outcome underscores a vital finding for this report: AI can generate "plausible" advice that remains entirely misleading if the underlying data retrieval is technically flawed.

Conversely, the final test regarding "Market Gap" criticality yielded the highest strategic value. The Agent proposed a comprehensive revision of the overall Tagging Strategy. This demonstrated significant analytical depth, as the Agent hypothesized that the root cause might involve poor question quality or overly generic categorization. Furthermore, the analysis of specific gaps, such as the 42.86% identified for "apache-spark," allows these metrics to be converted into targeted recruitment efforts, providing Human Resources with a clear, data-driven direction for talent acquisition.

Ultimately, the integration of a native AI Agent within Neo4j Aura represented the evolutionary peak of this project, transforming the database from a static archive into a dynamic decision-making consultant capable of natural language interaction. This exploration confirmed that the Agent's effectiveness is directly proportional to the quality of Prompt Engineering and the robustness of the underlying data modeling. While the system excelled at identifying community leaders and diagnosing critical Market Gaps, the process also highlighted inherent technical challenges, including vulnerabilities to case sensitivity and occasional discrepancies between logical intent and generated code syntax. These limitations emphasize that the Agent functions as a powerful assistant requiring constant human validation rather than a replacement for professional analysts. In summary, the implementation of the Agent has validated the strategic value of the graph, demonstrating how artificial intelligence can convert relational data into actionable insights to proactively drive community growth and health.